

*A pressure control device for
force application during optical
coherence tomography imaging
and control mechanism for low-
level laser therapy*

Brady Perkins, December 2025

Biodesign Lab, Asia University, Taichung City, Taiwan

1. Pressure control and recording device

Overview

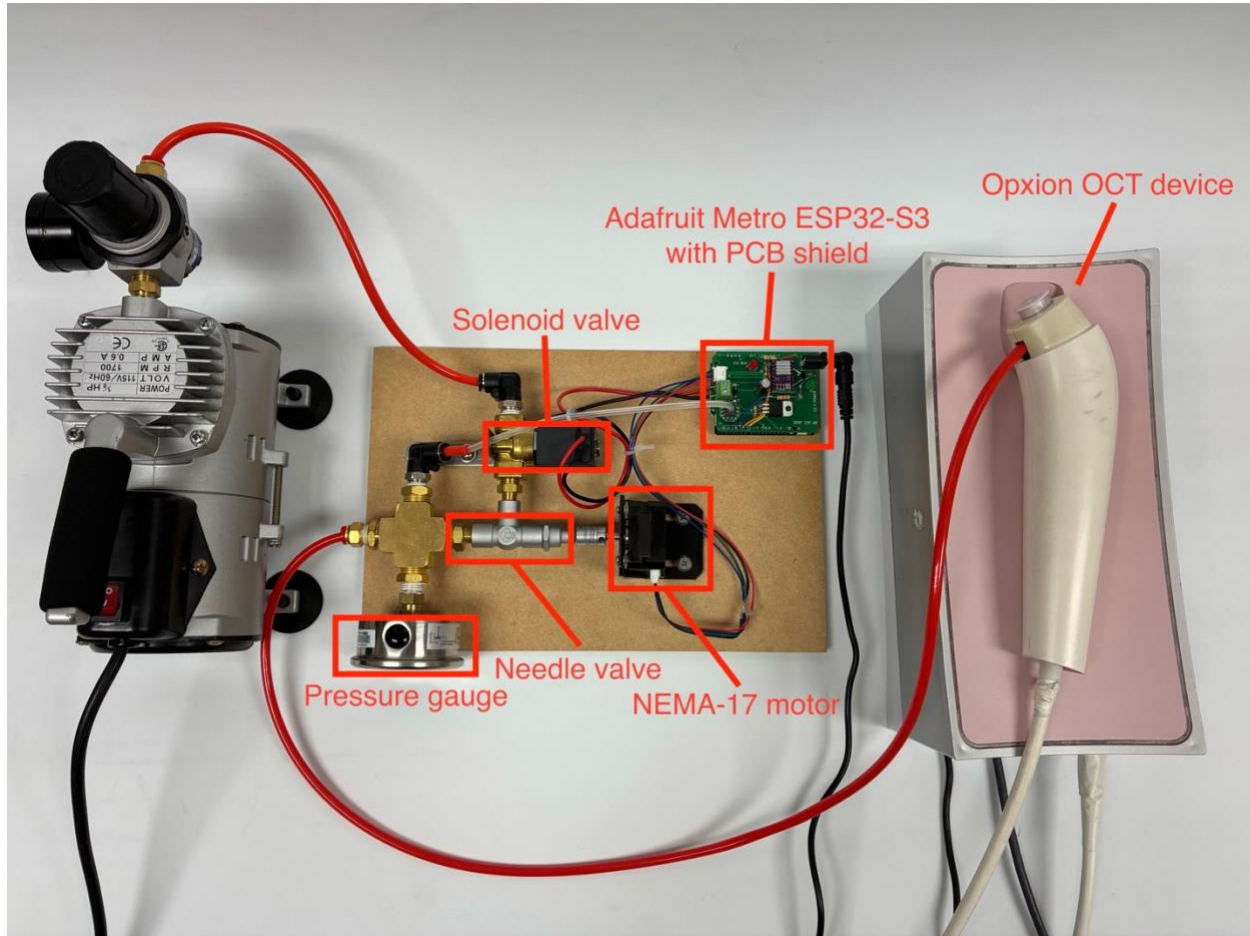


Figure 1: A labeled overhead view of the OCT pressure-control device.

A device has been created which uses a NEMA-17 stepper motor to control a needle valve used to adjust the airflow fed into an optical coherence tomography imaging device, as shown in Figure 1. A solenoid valve is also used to turn airflow on and off for certain air-burst applications.

Device control is accomplished using an Arduino IDE-compatible ESP32-S3-based microcontroller, the Adafruit Metro ESP32-S3. A custom shield PCB with Arduino UNO-compatible dimensions was also manufactured according to the schematic shown in Figure 2.

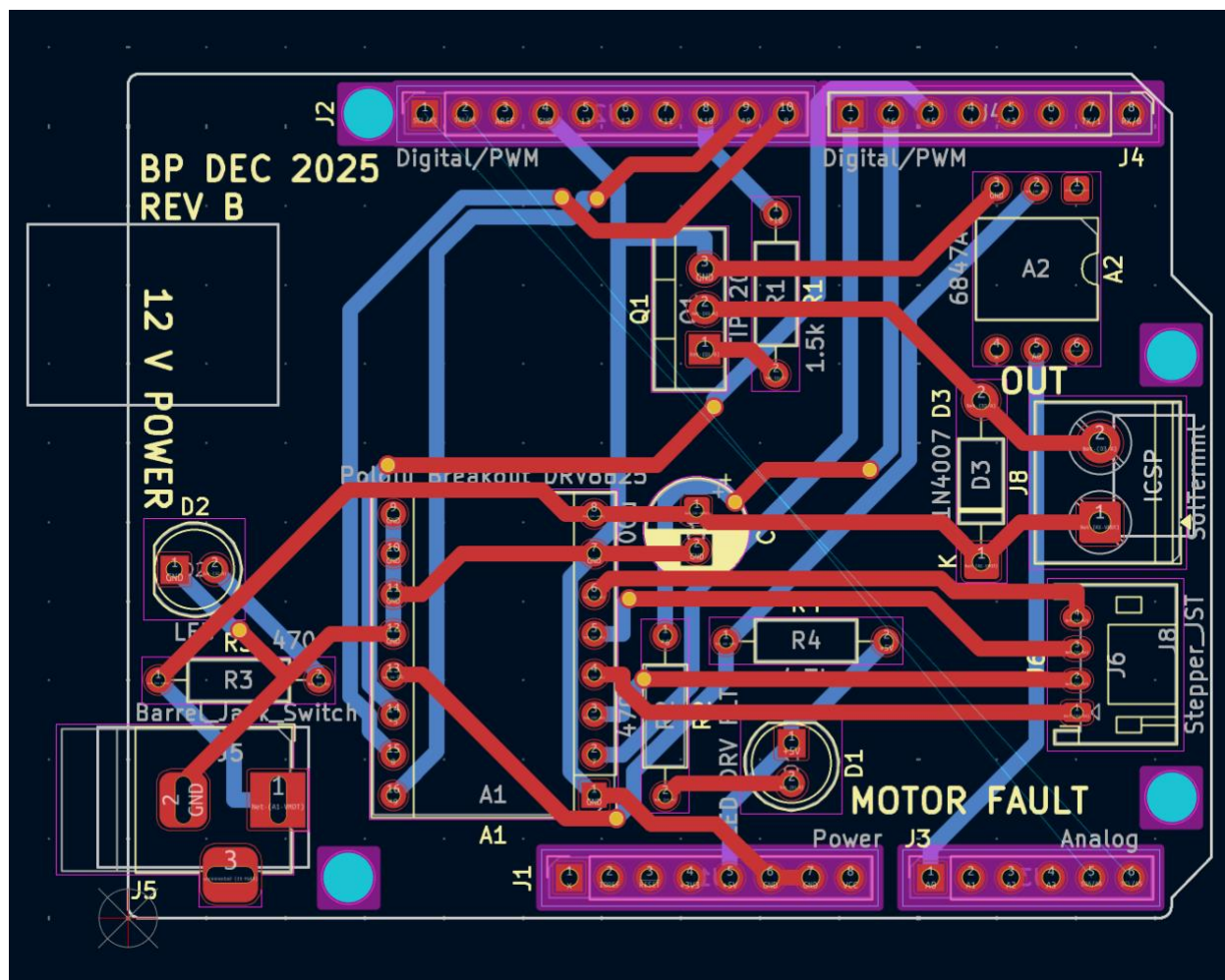


Figure 3: The two-dimensional KiCad model of the shield PCB (red traces lie on front layer, blue traces on back layer).

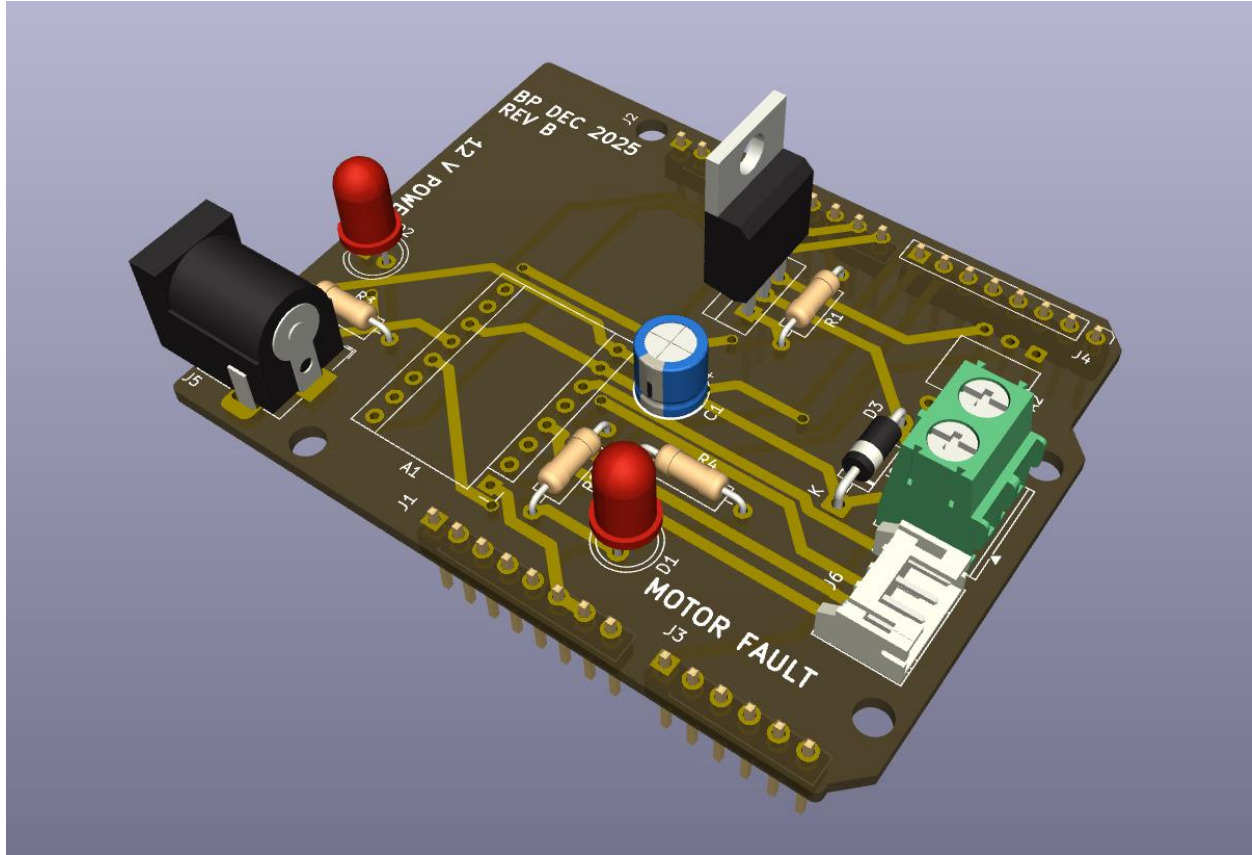


Figure 4: A three-dimensional KiCad model of the board semi-populated.

The solenoid and stepper motor are each powered using a 12 V DC power supply, which is fed to the PCB via the 2.1 mm barrel connector shown in the 3D model in Figure 4. An LED is used to visually verify 12 V power connection, and the 12 V power is fed directly to the VMOT and GND pins on the top-right of the DRV8825 (as shown in the pinout diagram in Figure 5) and to the solenoid through the TIP120 power transistor, which connects and disconnects the GND signal of the 12 V connection fed to the screw terminal connector on the right side of the board (to which the solenoid valve is attached).

The diode placed just before the screw terminal is a 1N4007 power diode to create a low-resistance path for flyback current (current of opposite polarity to the current supplied by the 12 V DC input) generated by the collapsing magnetic field of the inductor in the solenoid valve upon its power-off.

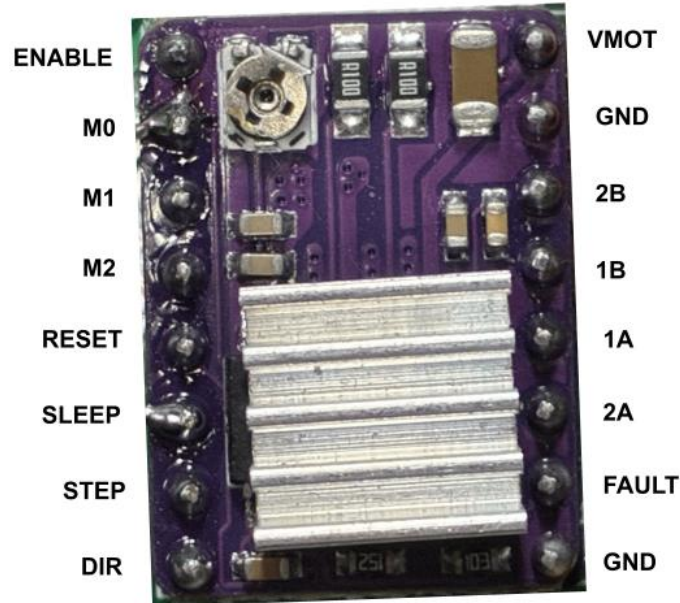
Pololu DRV8825 pinout

Figure 5: The pinout of the Pololu-style DRV8825 stepper motor driver.

Other connections also shown in Figure 5 include the motor coil connections: 2B, 1B, 1A, and 2A, as well as the “FAULT” signal pin, which is activated any time the H-bridge FETs driving current to the motor coil connections are disabled, a logic-level GND pin, ENABLE, RESET, SLEEP, STEP, DIR, and the (here unused) fractional stepping pins, M0, M1, and M2.

Signals of the DRV8825 that this pressure control device utilizes include the necessary STEP and DIR – which give instruction for the motor to step when pulsed and change the motor direction based on high or low input, respectively – as well as the SLEEP pin used to disable the H-bridge FETs when the motor does not need to receive current (while it is not moving), and the FAULT pin, which is used to read whether the H-bridge FETs have been disabled while the motor should be turning. All other pins – ENABLE, M0, M1, M2, and logic GND – are connected to ground.

The VMOT and GND motor power inputs (12 V DC) also have a 100 μ F capacitor placed across them in order to ensure a clean and constant motor power supply input to the driver.

Device and microcontroller connections

Microcontroller general-purpose input/output (GPIO) pins are used to communicate between the devices housed on the PCB and the microcontroller which sits underneath the PCB. The pin connections of each device are as given in Table 1.

Table 1: GPIO-device connections.

Arduino UNO-compatible GPIO pin	Device connection on PCB
5	DRV8825 SLEEP
6	MOTOR FAULT LED
7	DRV8825 FAULT
8	DRV8825 STEP
9	DRV8825 DIR
10	Solenoid (TIP120 base)
A0	Pressure sensor output (6847A pin 5)

Pins 11-13 remain unused to avoid interference with the SPI interface on many Arduino UNO-compatible microcontroller designs. All ground connections on the Arduino are used as ground connections on the shield PCB, and the highest positive-voltage output (the voltage output pin closest to ground in the power row of Arduino UNO-compatible designs) is used as the logic-level voltage connection (+5 V on Arduino UNO and +3.3 V on ESP32 platforms) for the DRV8825 and 6847A pressure sensor.

The Adafruit Metro ESP32-S3 microcontroller was purchased from Digi-Key Taiwan (Digi-Key Electronics, Thief River Falls, Minnesota, United States), while initial PCB prototypes as shown in Figure 1 were manufactured by Yuan Guang PCB-togo (Yuan Guang PCB-togo Electronics, Inc., Taoyuan City, Taiwan) and later PCB units based on the schematic shown in Figure 3 manufactured by PCBway (Shenzhen JDB Technology Co., Ltd., Shenzhen City, Guangdong Province, P.R.C.). Electronics components were purchased at Jin Hua Electronics (今華電子, Taichung City, Taiwan).

Airflow within the device

The path of the air through the device begins at the air intake, which is connected to a positive-pressure air compressor through a pressure regulator of a low pressure (<100 kPa) set by the user. The air first travels through the solenoid valve, then through the needle valve to a three-way splitter that acts as a pressurized chamber connected to both the 6847A electronic analog pressure sensor module, a mechanical pressure gauge, and the output tube to be connected to the OCT imaging device.

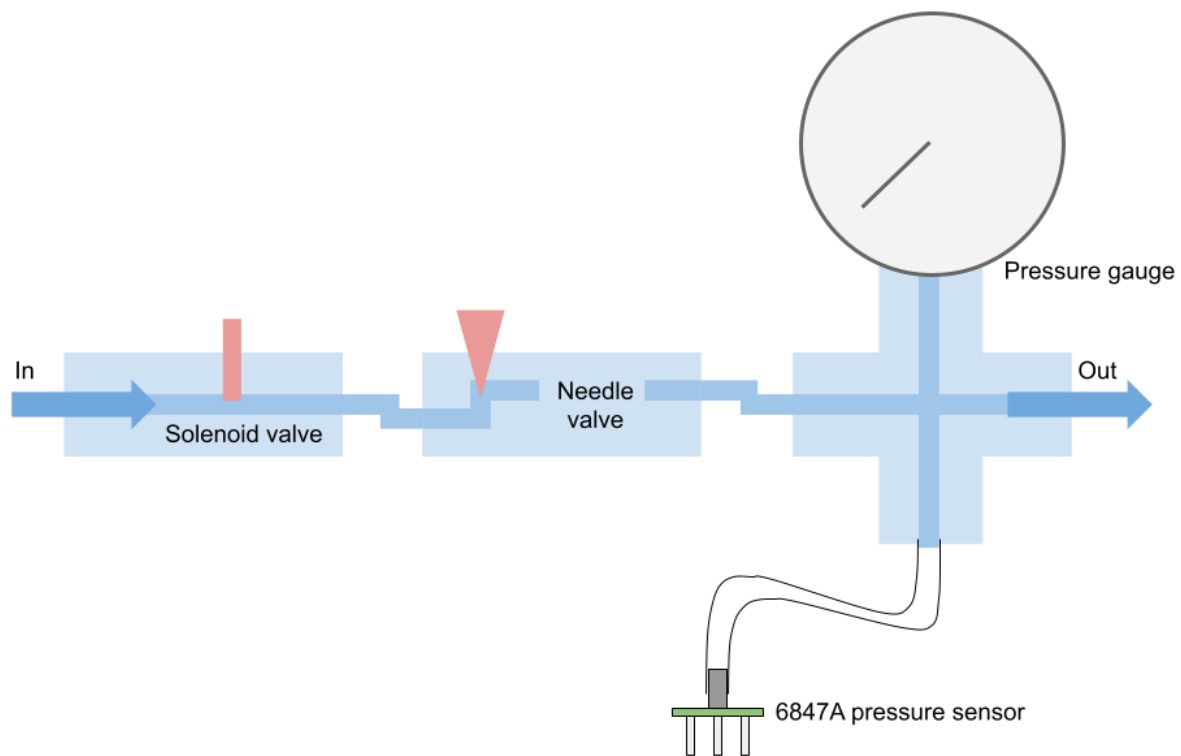


Figure 6: A map of the airflow through the device.

The flow of air through the device is given in a simple map in Figure 6, with airflow control joints represented in red and arrows representing the direction of flow. The solenoid valve has two states: fully open and fully shut, while the needle valve uses a varying-width needle to adjust the airflow with precision.

Airflow control mechanical connections

The stepper motor and needle valve are connected using an axle coupler of internal diameter 5 mm. The diameter of the motor axle is 5 mm exactly, and the diameter of the needle valve axle is 4.5 mm. The additional volume between the needle valve axle and coupler are filled using solder.

The connections between each of the airflow devices are measured in United States imperial units (inches), while the tubes are measured in metric units (millimeters).

The internal diameter of the screw connections on the solenoid valve are 0.375 inches, the screw connections on the needle valve are 0.25 inches, and screw connections on the three-way splitter are 0.5 inches. Screw size adapters are used to interface each component.

The screw diameter of the tube connections to the input, output, and pressure sensor are each 0.375 inches, and the diameter of the connected tubes are 4 mm with a wall thickness of 1 mm (making for a total outer tube diameter of 6 mm).

The smaller tube connection to the pressure sensor has an outer diameter of 4 mm with a wall thickness of 1 mm, for an inner diameter of 2 mm.

The screw diameter of the pressure gauge connected to the three-way splitter is 0.5 inches.

Each of the hardware components were purchased at ALD Hardware (振宇五金, Taichung City, Taiwan), except for the needle valve and solenoid valve, both purchased at Wen Jun Hardware (文俊五金, Taichung City, Taiwan).

Before the air input is a pressure regulator connected to the compressor, model Mindman MAR 300-8A (Mindman Pneumatics, Taipei City, Taiwan).

Data input and output

Data input and output is accomplished using the microSD card slot on the Adafruit Metro ESP32-S3 board.

On startup, the microcontroller reads from two files in a folder on the SD card called “sys”: calibration.csv and wifi.csv.

The file calibration.csv contains the values of the average of three zero-points of the 6847A pressure sensor, used as the reference for 0 kPa, as well as the average value of three high calibration points taken at 20 kPa. The values are separated by a comma, in this format: 500,1000. Values are read from the pressure sensor's output pin via the Arduino analogRead function, so in this example, the reference for 0 kPa is analogRead result value of 500, while the reference for 200 kPa is an analogRead result value of 1000.

The file wifi.csv contains the authentication information for a wireless network, used so that the Arduino can retrieve the date & time, used for dating trial data when it is saved to an output file on the SD card.

While running, two additional folders on the SD card are used for input and output of information. These are "in" and "out". The "in" folder contains CSV files with two columns: time, in milliseconds, on the left and motor position, in degrees, on the right. These are treated as programs which can be loaded on the microcontroller. The "out" folder is the folder which dated CSV output data files are placed in after a trial is run. These CSV files contain four columns, in order, time in milliseconds, output force in grams, tube pressure in kilopascals (kPa), and motor position in steps, where one step is equal to 1.8 degrees of rotation.

Names for these files are generated using the date retrieved from the Internet in the format YYYYMMDD_HHmmSS_oct.csv, and the initial date used is 2025-01-01 00:00:00 if unable to retrieve a time from the Internet (for example, if the network is not connected to the Internet or if the microcontroller is unable to connect to the network).

Communication between the user and the microcontroller is accomplished over a serial terminal at 115200 baud, connected to a computer via a USB cable.

Device calibration

Device calibration is accomplished over the serial connection between the ESP32 microcontroller and a computer. A calibration process is begun by entering "cal" through the terminal and following on-screen prompts to take three pressure sensor measurements for a high calibration point and zero calibration point. Pressure values read by the sensor are not absolute, but instead relative to atmospheric pressure.

The high calibration point is set to a pressure of 20 kPa, and the user is asked to adjust the position of their finger on the end of the outlet tube until the pressure output reads 20 kPa, then to enter the word “next” into the serial terminal. After the measurement of three calibration 20 kPa calibration points, the user is asked to remove their finger from the end of the outlet tube so that the device can take three zero points at atmospheric pressure (0 kPa).

Software control

Input to the device is given via an input CSV file to be interpreted by the software on the microcontroller. The CSV file contains two columns, with each row representing a point in the movement of the motor: a timestamp (in milliseconds) in the left column, and a motor position (in degrees) in the right column. An example is given in Table 2.

Table 2: An example CSV with motor program input data.

0	0
500	0
1500	300
2000	300
3000	0
4000	0

Upon serial connection, the user types “load”, followed by the name of the file as it is listed in the SD card’s /in/ folder, without file extension: for example, for the file /in/program.csv, the user enters “load program”. When a program is loaded, the CSV is converted to a series of instructions to be processed by a program interpreter, which specify the amount of time to record for (in 25 Hz frames to match the recording rate of the Opxion OCT device), the amount of time to keep the solenoid open for (in milliseconds), followed by pause, motor velocity, and motor position instructions. The corresponding program to the example in Table 2 is:

r100,s4000,t500,v300,m300,t500,m0,t1000

This program specifies to record a CSV file for 100 frames (4 seconds), to open the solenoid for 4000 milliseconds (so that the solenoid is open for the same time period as the recording), followed by a 0.5-second pause timer before the motor velocity is set to 300 degrees per second

and moved to position 300 degrees. Another 0.5-second pause timer precedes an instruction to move the motor back to position 0 degrees and to wait a final period of 1 second.

To run the program, the user enters “run” into the serial terminal.

The initial recording and solenoid time are calculated by the CSV interpreter based on the period from the first timestamp listed in the CSV file to the final timestamp. The maximum allowed motor speed is 400 degrees per second.

Upon running of the program, a program interpreter reads these serial instructions and moves the motor and solenoid accordingly. Recording and solenoid opening instructions are non-blocking, so that recording and solenoid open time can occur alongside motor movement. The timer instruction is blocking, to allow for periods during the recording where the motor should not move, for example, to generate buffer time at the beginning of the program to allow for delay before the start of the OCT recording.

The OCT recording and motor programs are begun asynchronously, since the OCT device relies on control software shipped by its manufacturer which does not allow for remote instruction from other running programs. CSV line recording is done at 10 Hz, while OCT frame recording is done at 25 Hz.

Programs in the comma-separated serial format can also be entered directly after the “run” command. For example, if the solenoid must be turned on and off while the needle valve is open, this cannot be accomplished using the CSV interpreter. A program must be entered directly after “run”, for example:

```
r100,s500,t1000,s500,t1000,s500,t1000,s500,t1000
```

This example program records for 100 frames and opens the solenoid for 0.5 seconds over four 1-second time intervals, causing the solenoid to toggle on and off every 0.5 seconds.

Upon completion of a record instruction, the data is saved to the SD card’s /out/ folder with its dated filename and four columns containing time, output force in grams, tube pressure in kPa, and motor position in steps (to avoid using the compute necessary for float math to convert motor position in steps to motor position in degrees during data output).

Output force in grams is calculated using a simplified form of the Bernoulli equation which assumes a horizontal tube network (no change in gravitational potential) and a tissue sample less than one tube diameter (less than 4 mm) away from the end of the outlet. This is given in Equation (1).

$$F = 2A\Delta P \quad (1)$$

Where A is equal to the cross-sectional area of the outlet tube in square meters, and ΔP is equal to the pressure difference between the inner tube network and the outer atmosphere.

This simplified relationship is derived from the Bernoulli equation:

$$F = P_{atm} \cdot (A) + \rho QV_{out} \quad (2)$$

In Equation (2), V_{flow} is equal to the velocity of the air leaving the tube, and ρ is equal to the density of the fluid (air). Q is the volumetric flow rate of the fluid, equal to the cross-sectional area of the tube times the velocity of the fluid.

$$Q = AV_{out} \quad (3)$$

Substituting Equation (3) into Equation (2) gives Equation (4):

$$\begin{aligned} F &= P_{atm}A + \rho AV_{out}^2 \\ F &= A(P_{atm} + \rho V_{out}^2) \end{aligned} \quad (4)$$

With a horizontal network of tubing, Equation (5) is assumed:

$$P_{in} + \frac{1}{2}\rho V_{in}^2 \approx P_{out} + \frac{1}{2}\rho V_{out}^2 \quad (5)$$

If the input velocity is very low (outlet of air pressure regulator and tubing network inlet are at about the same pressure, so the pressurized tube can be approximated as a large air reservoir):

$$\Delta P = P_{in} - P_{atm} \approx \frac{1}{2} \rho V_{out}^2$$

$$\rho V_{out}^2 = 2\Delta P$$
(6)

Substituting Equation (6) back into Equation (4) gives Equation (7):

$$F = A(P_{atm} + 2\Delta P)$$
(7)

Since the pressure at the outlet, P_{atm} , is atmospheric pressure, P_{atm} can be approximated as insignificant compared to ΔP (the pressure inside the tubing network).

This leaves Equation (1):

$$F = A(2\Delta P)$$

$$F = 2A\Delta P$$
(1)

In Equation (8), A is in units of square meters, and ΔP is in units of kilopascals (kPa). The output force in this equation is in units of Newtons, which are divided by a factor of 1000/9.81 for conversion from Newtons to grams. A is calculated as πr^2 , where $r = 0.002$ meters (2 millimeters, half of the tube's 4 mm diameter), for a constant result of 0.00001257 m². ΔP is calculated by scaling the voltage reading output value from analogRead to 0 kPa and 20 kPa calibration points using Equation (9).

$$\Delta P = \frac{v}{k} \cdot 20$$
(8)

In Equation (9), v is the result from analogRead (the reading from the 6847A pressure sensor through a microcontroller analog input pin), and k is the difference between the analogRead

values of the high calibration point (at 20 kPa) and the zero calibration point, while 20 is the 20 kPa scaling factor.

A note about the choice of microcontroller: the software for the pressure control device addresses each function asynchronously from inside the main loop in order to enable multitasking on any microcontroller. However, in addition to wireless Internet and removable storage capability, the Adafruit Metro ESP32-S3's additional speed compared to an Arduino UNO helps to prevent slowdown, aiding motor movement accuracy during output CSV writing and force calculations.

Low-level laser therapy control device

Overview

The control device for low-level laser therapy application reuses the PCB design from the pressure control device, making use of the 12 V DC output originally for the solenoid valve for the laser power supply. The device setup is given in Figure 7.

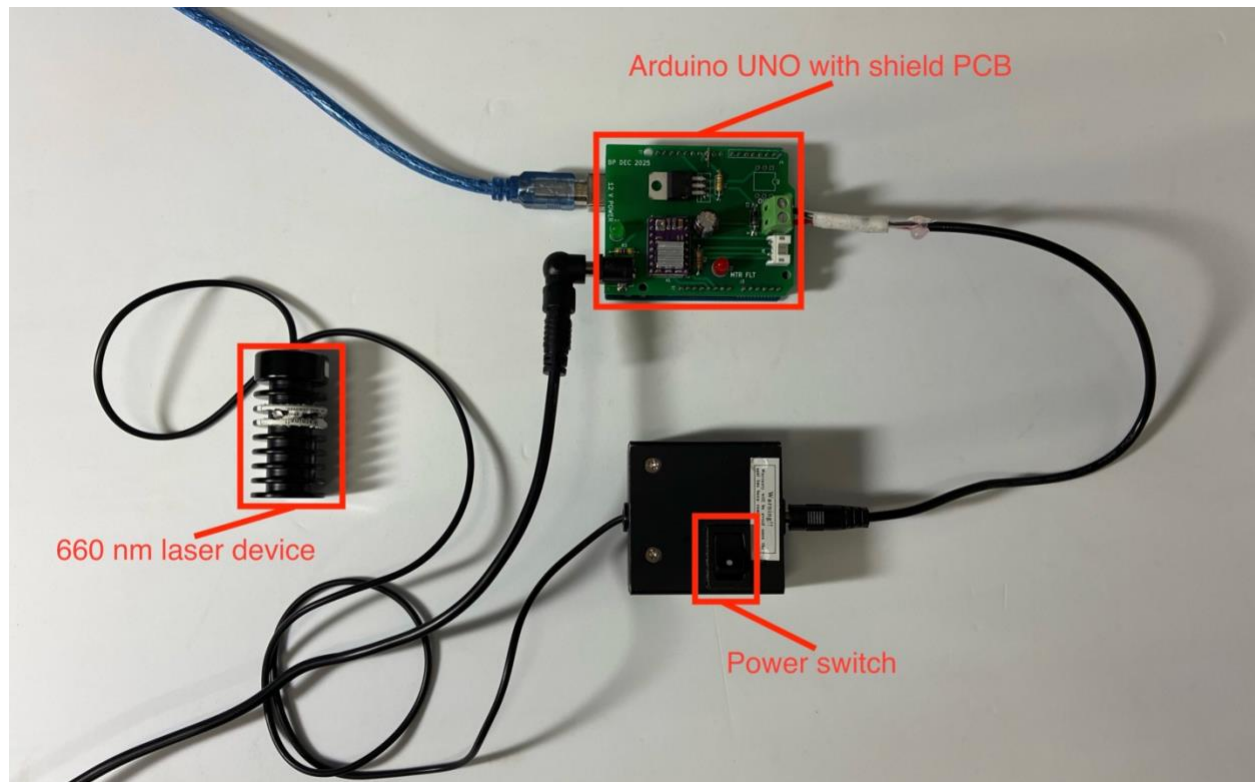


Figure 7: Low-level laser therapy control device.

The laser used is a 660 nm-wavelength device with a power input box that uses a DC voltage regulator to convert a range of higher-voltage input power between 8 and 30 V into the 3.3 V power used by the laser device, as well as containing a power switch used to turn the device on and off.

The microcontroller utilized in this device is an Arduino UNO instead of the Adafruit Metro ESP32-S3 board used for the pressure control device, since removable storage, Internet capability, and additional speed are not necessary.

Laser control/user interface

Laser control is also done via a serial terminal, running at 9600 baud on the Arduino Uno instead of the 115200 baud utilized by the ESP32 microcontroller. The user can only enter commands into the serial terminal in one format:

nn,dd

where *nn* is the desired flashing rate of the laser (in Hz), and *dd* is the desired duty cycle of the laser refresh (in percent). The default setting is:

1,50

which corresponds to a refresh rate of 1 Hz and a 50 percent duty cycle.

Trials are conducted by deciding the amount of time that the laser should remain on for, and this time is tracked using an external stopwatch which is begun as the power switch on the laser box as shown in Figure 7 is turned on, and where the power switch is turned back off once the stopwatch reaches the set amount of time.

Device resources

Pressure control device (GitHub):

<https://github.com/bap10357/llt-controller>

Low-level laser therapy device program (GitHub):

<https://github.com/bap10357/oct-pressure-device>

Device quick-start

How to set up

Place the device on a flat surface and plug the labeled power cord into a power outlet (best if using a power strip, since other devices must be used at the same time), as shown in Figure 8.

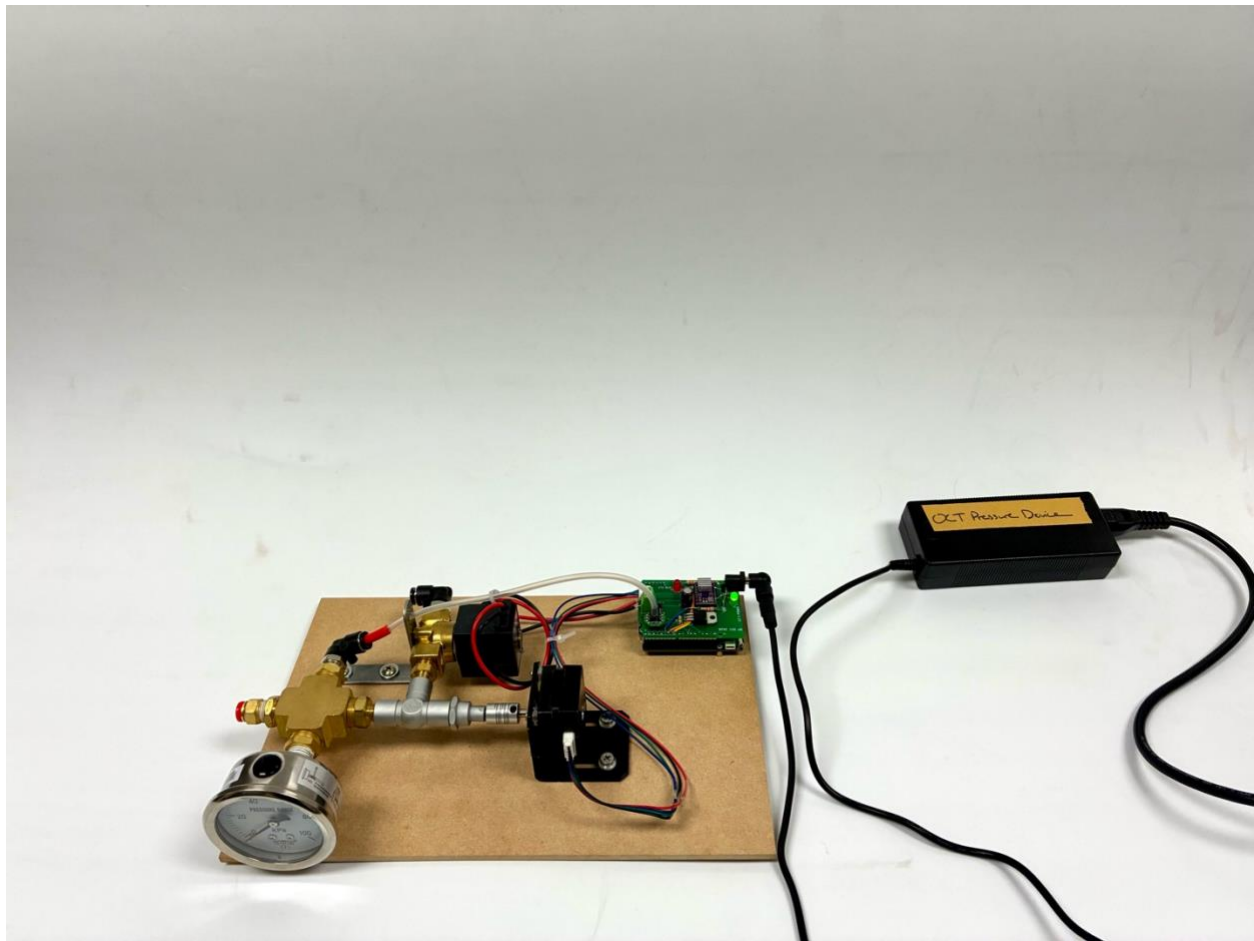


Figure 8: The pressure device plugged in to power.

If using a switched outlet or power strip with switched outlets, it may be best to turn the device off until ready to test to restrict current flow to the high-power components unless necessary.

Next, connect the designated air compressor to the device (shown in the picture is the positive-pressure compressor attached to an air pressure regulator set to a maximum output pressure of about 100 kPa). Use the short red tube to attach the output from the pressure regulator to the input on the device (the black tube connection as shown in the top-left of the board in Figure 9).

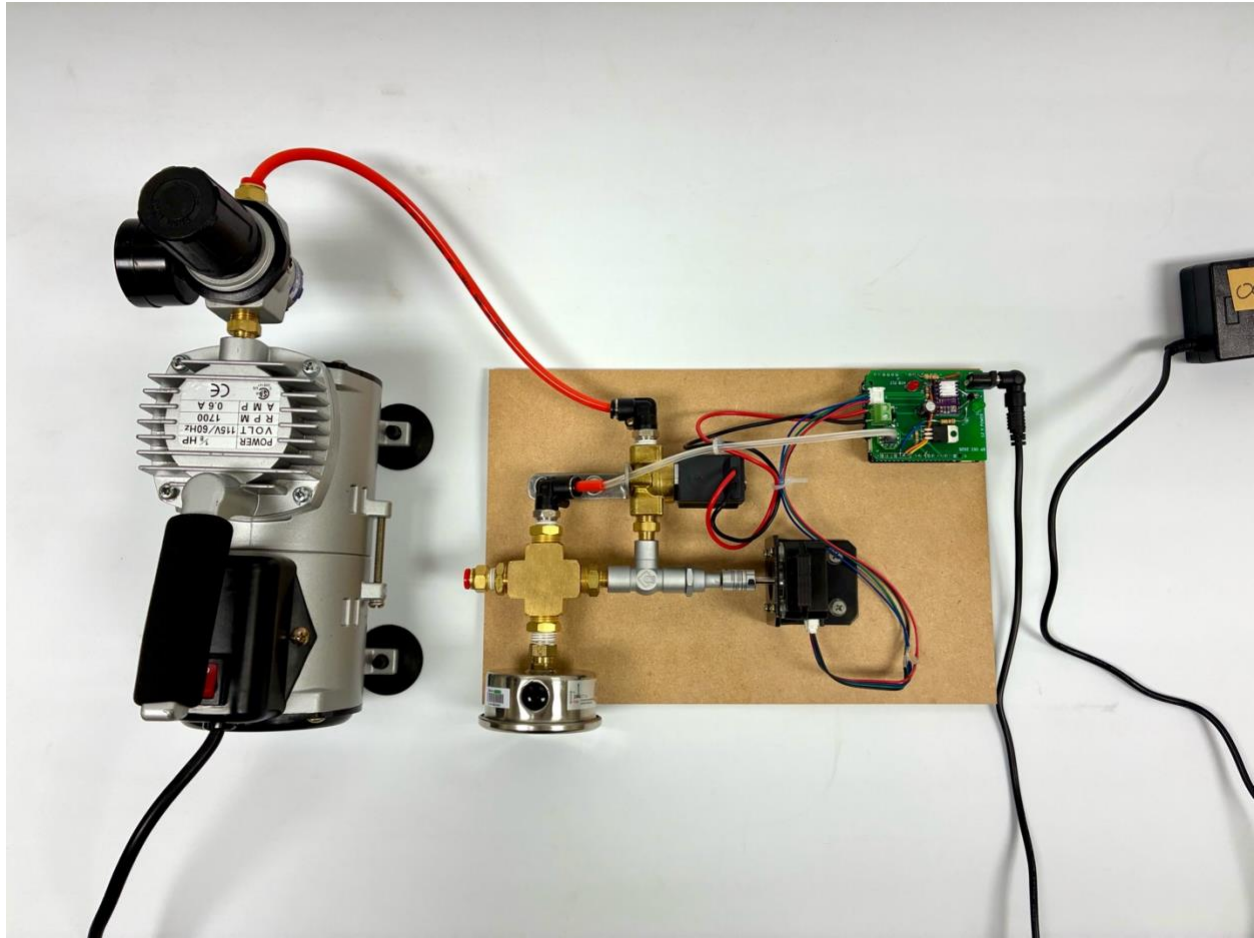


Figure 9: The pressure device connected to the air compressor.

Then, use the longer red tube to attach the outlet of the device (the red connector at the end of the splitter to the left of the gauge, shown in Figure 9). The cap of the end of the OCT wand that places over the standoff cannot be attached to the device: the hole left for the red tube to connect to the inside of the wand is not large enough. With the cap removed but the standoff at the end of the OCT wand left in place, connect the red tube to the OCT wand, as shown in Figure 10.

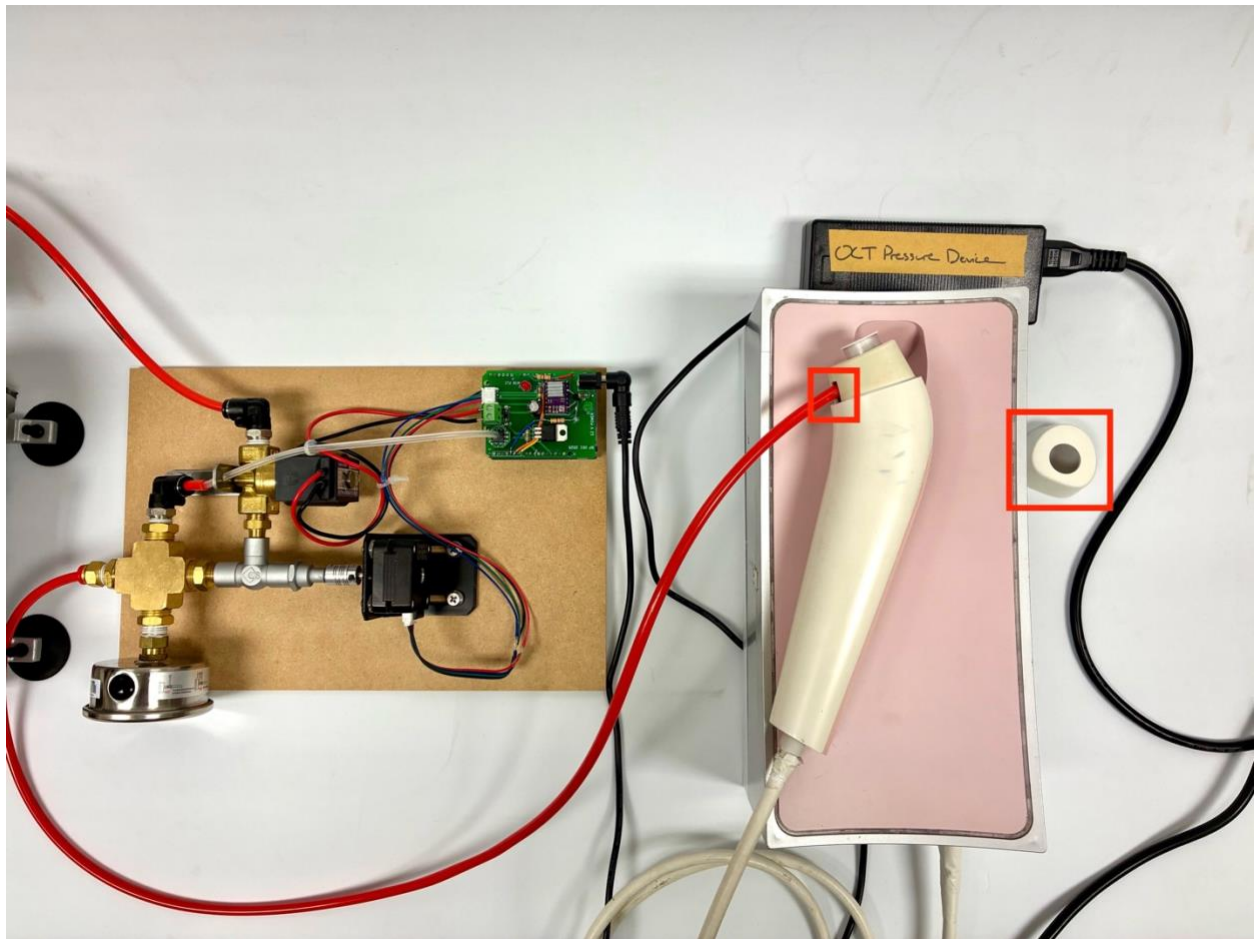


Figure 10: The OCT wand connected to the device using the red tube, with the tube and removed cap in red.

Following this, connect both the air compressor and OCT to power and switch the two outlets on if using switched outlets.

Then, connect the USB cord to the bottom of the OCT device and connect its other end to a computer running Arduino IDE and the Opxion OCT software (at Asia University, the lab MSI laptop).

Open Arduino IDE and the ESP32 microcontroller should connect. After startup, optionally, you can also open the serial plotter by clicking the leftmost button at the top-right part of the blue ribbon (which will label itself “serial plotter” on hover), as in Figure 11.

If the device does not connect, first ensure that “Adafruit Metro ESP32-S3” is selected in the drop-down box at the top-left of the Arduino IDE window (as shown in Figure 11). If

it does not appear, unplug and re-connect the USB cable.

Select “115200 baud” as the serial connection speed above the right side of the terminal area.

If the serial terminal area is unavailable, click the “Serial Monitor” button to the right of the “Serial Plotter” button to re-open the serial pane.

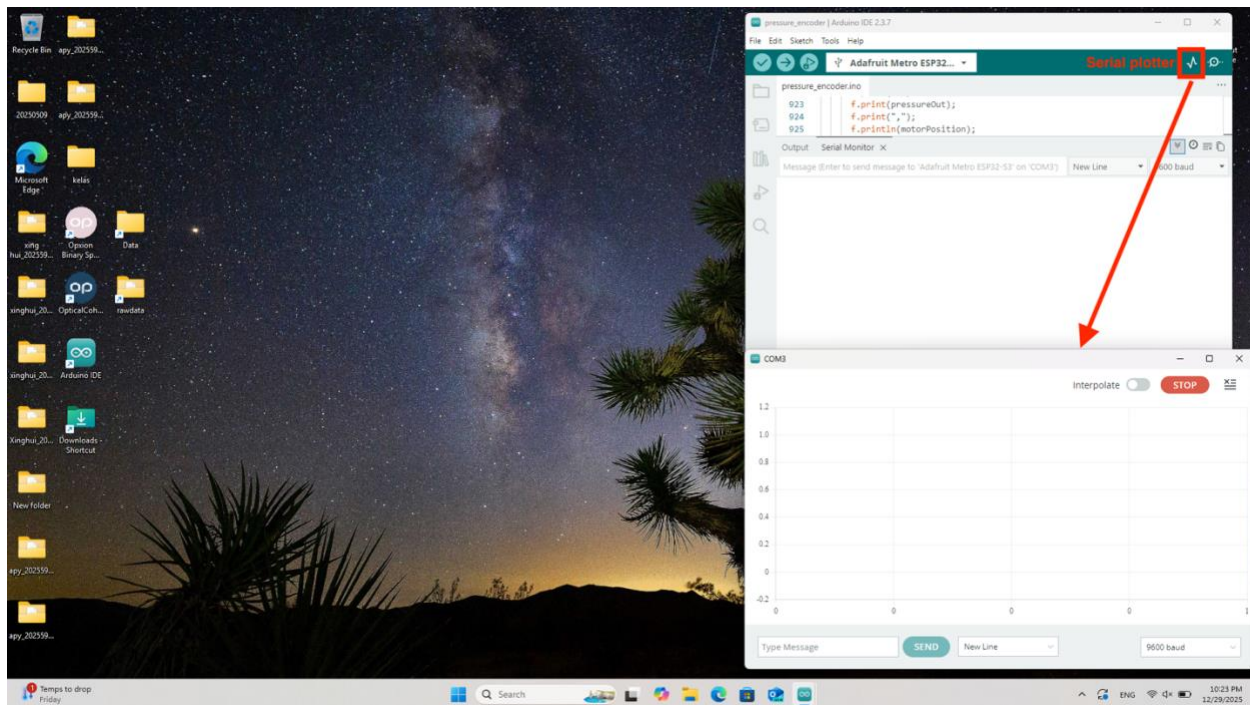


Figure 11: A Windows desktop with Arduino IDE open and the serial plotter labeled.

Optionally, you can also press the reset button just to the left of the USB plug on the Adafruit Metro ESP32-S3 board, shown in Figure 12, to watch the startup sequence and verify successful initialization of the SD card and wireless network, shown in Figure 13.

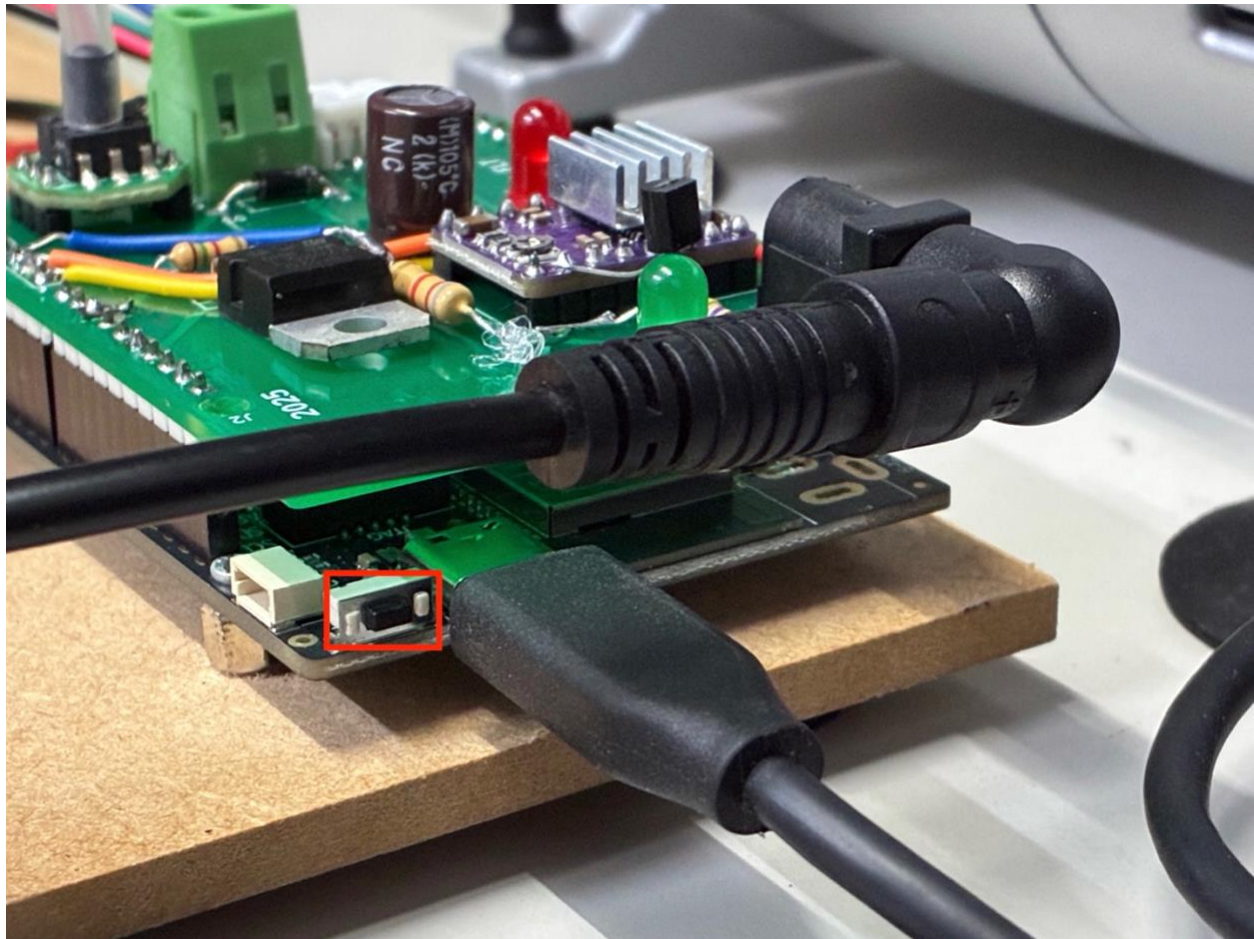


Figure 12: The reset button on the Adafruit Metro ESP32-S3 relative to the rest of the pressure device, in red.

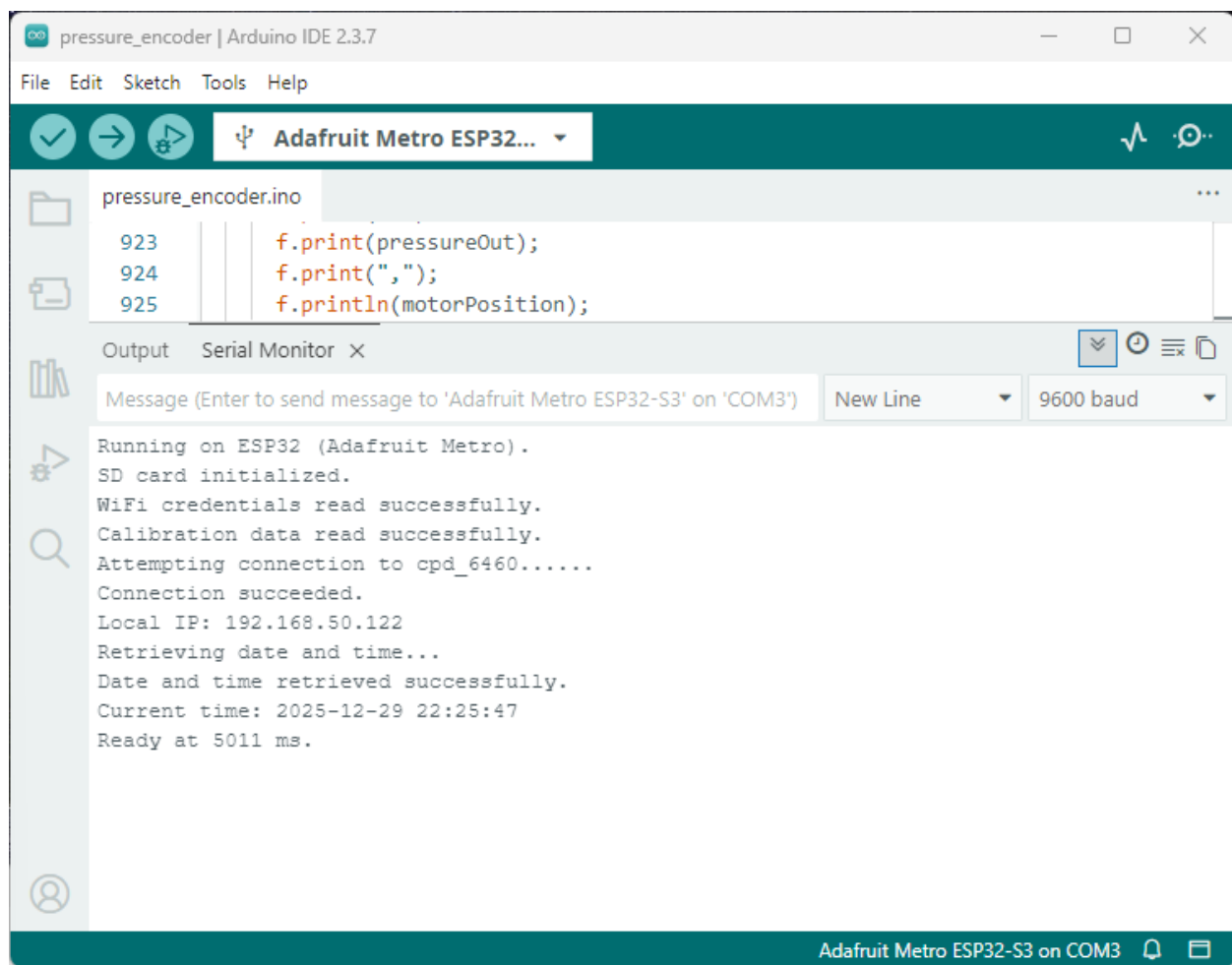


Figure 13: A successful startup sequence of the Adafruit Metro running the control software.

If the SD card is unable to connect, first, check that the SD card is fully inserted. If it is, remove and re-insert.

If the WiFi is unable to connect or if the SSID and password are incorrect, type “wifi ssid,password” at the serial prompt to enter a new SSID and password. Press the reset button again as shown in Figure 12 to re-attempt startup and wireless network connection.

After successful startup of the ESP32 device, the Arduino serial connection, and the OCT software, the desktop should be arranged as in Figure 14, with all windows visible at the same time.

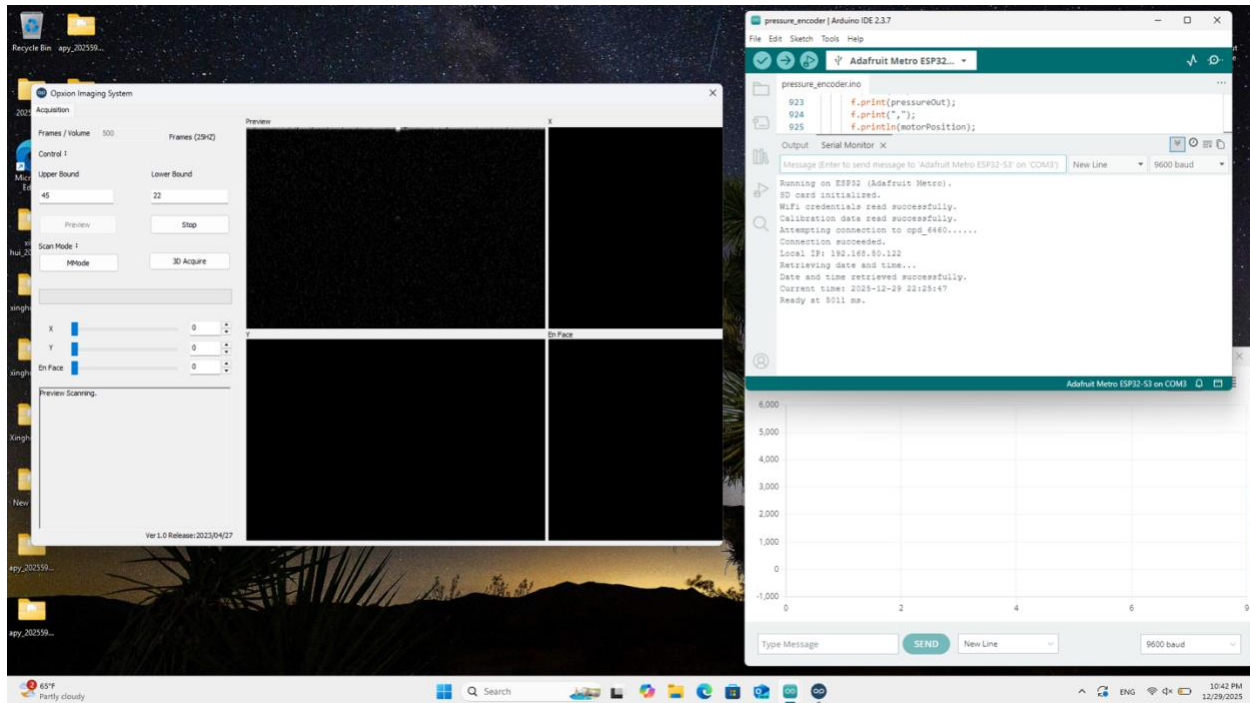


Figure 14: The desktop with OCT software and Arduino IDE serial monitor and plotter arranged together.

To load a program, type “load *programname*” to load the file */in/programname.csv* from the SD card. For example, to load the default program, type “load default” to load the file */in/default.csv* (a basic program that moves the motor from 0 – 720 degrees five times).

Before the trial begins, it may also be best to compare the set time lengths between the OCT recording and the program loaded into the interpreter on the microcontroller. Type “list” to see the list of commands converted from the input CSV file. For example, for */in/default.csv*:

r500,s20000,t1000,v400,m720,m0,m720,m0,m720,m0,m720,m0,m720,m0,t1000

In the first two commands, you can see the instructions to run a data recording (“r”) and to open the solenoid valve (“s”). These are values calculated by the CSV interpreter based on the entire length of the program defined in the input CSV file.

The input argument to the “r” command is the length of the program in OCT frames (at 25 Hz), while the input argument to the “s” command is the program length in milliseconds. Compare these values to your expectation and to the recording length specified in the OCT software (the default value is 500 frames, matching with the length of */in/default.csv*).

Just before the trial begins, it is also helpful to pre-type the word “run” into the Arduino serial monitor (the input box labeled “Message (Enter to send message to ‘...’”). Do not press the enter key yet.

To begin a trial, first press the “MMode” button in the OCT software window.

The OCT software will wait a moment before flashing the preview window and beginning its recording. This will take different amounts of time depending on the trial, so as soon as the “MMode” button is pressed, move your cursor back to the Arduino IDE window and the input box where you just typed “run”. When the OCT preview window flashes, press the enter key to input the “run” command and begin the OCT pressure control software, which will begin recording immediately.

When the software is done, you will see both a record of the deformation over time from the OCT software, as well as the graph of motor position and force over time if you opened the Arduino IDE serial plotter, as shown in Figure 15.

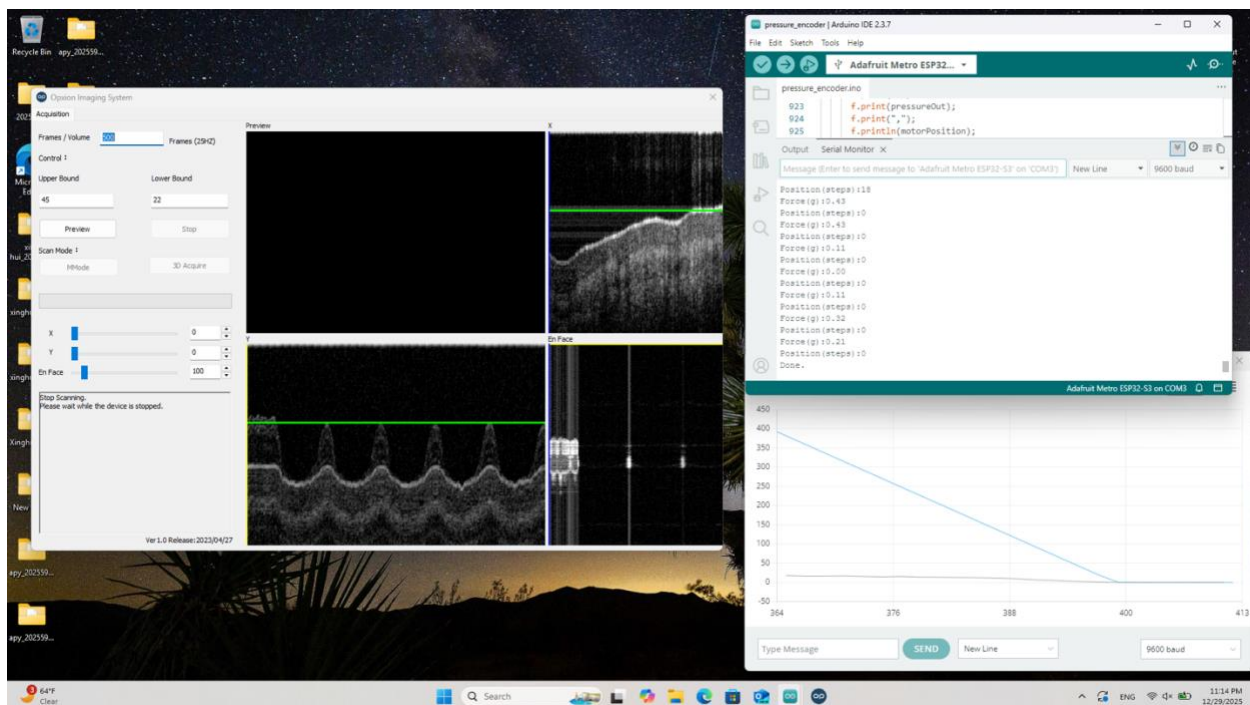


Figure 15: The result in the OCT software and in Arduino IDE after a trial has been run.

After the trial is complete, the CSV results from the pressure device are saved to the /out/ folder on the SD card, with the dated filename YYYYMMDD_HHmmSS_oct.csv.

Occasionally, during an OCT trial, when the tip of the OCT and the tissue or skin are placed too close to each other, the air pressure at the tip of the OCT will build between the skin and the OCT imaging sensor. This will cause the layer of the skin to vibrate and the sensor to be unable to detect an image until the air pressure reduces, leaving an overall output similar to what is shown in , where the image becomes blank when pressure begins to increase.

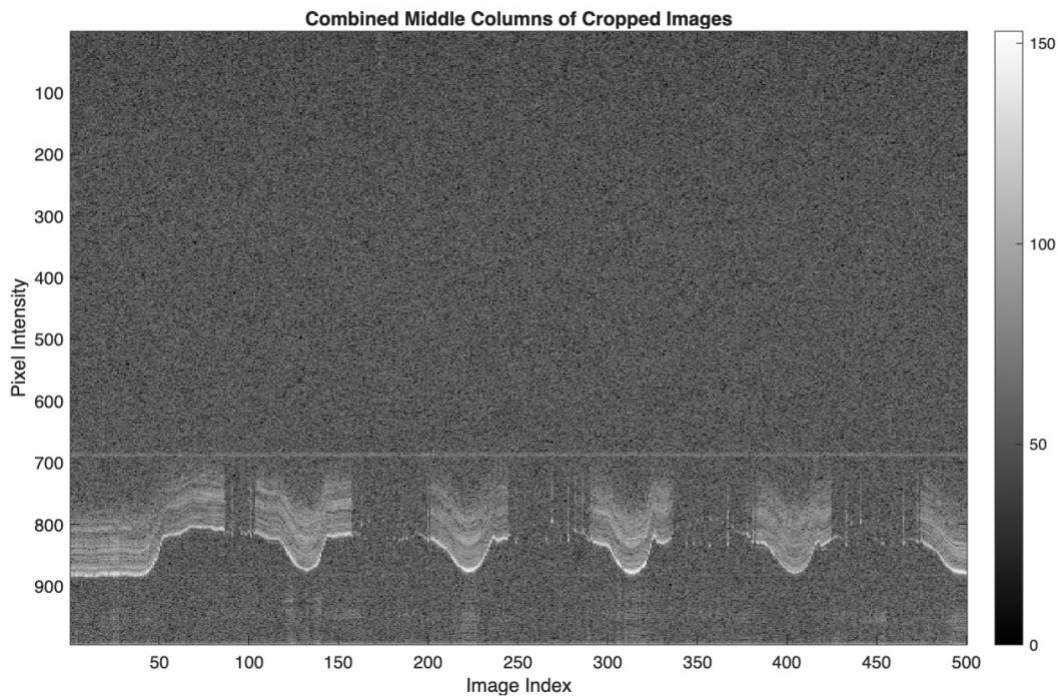


Figure 16: An example OCT image figure with the blanking problem.

To avoid this, leave a slight angle between the tip of the OCT wand and the tissue being imaged, enough to always leave a very small path for air to escape as pressure increases.

After an OCT trial is complete, to run the laser device (for example, after a control trial), power off or unplug the Adafruit Metro ESP32-S3 board from power and unplug it from USB on the laptop. Connect the Arduino from the laser device to the laptop with a USB cable and plug the power supply from the pressure device in to the barrel jack on the shield PCB on the laser device's Arduino, as shown in Figure 17.

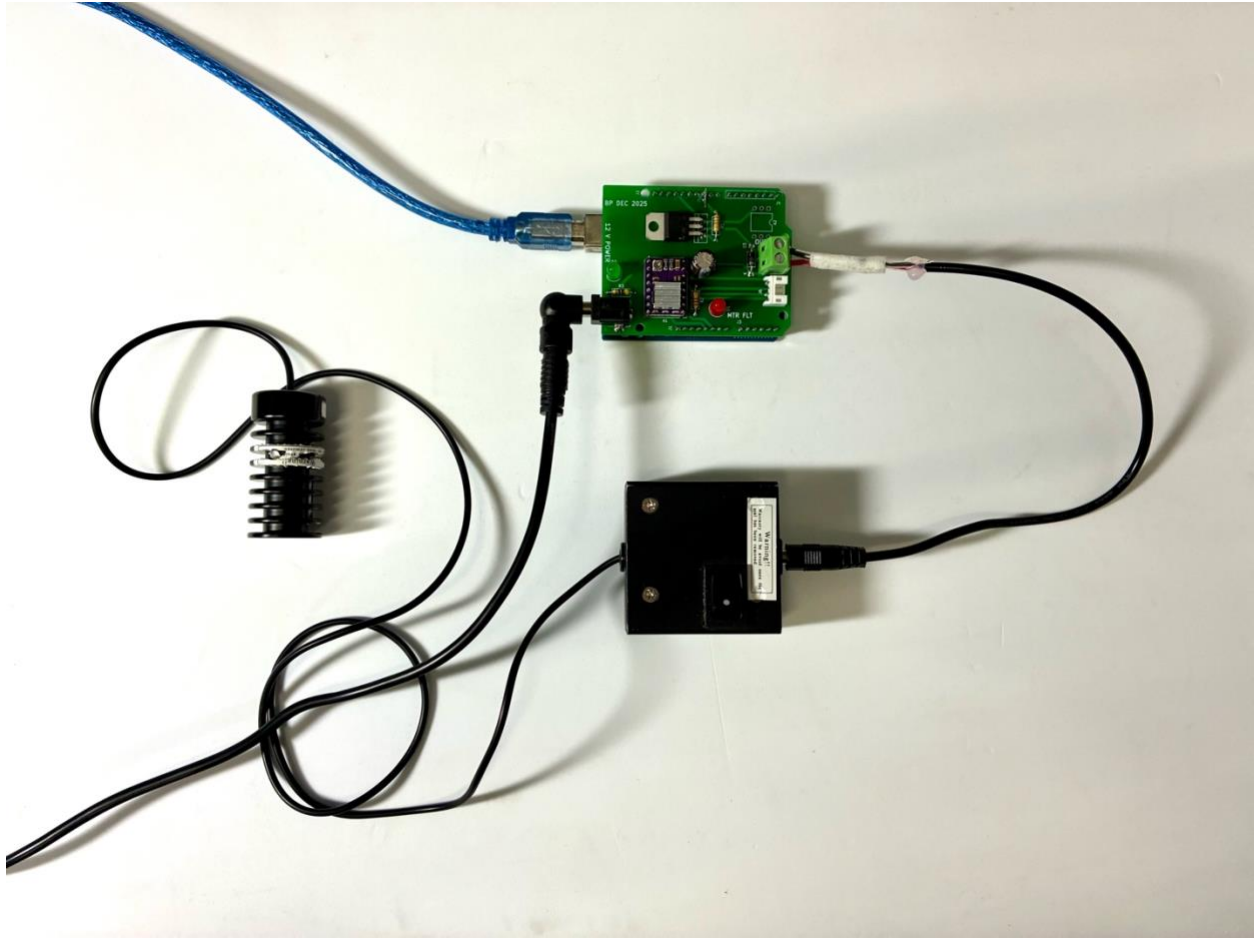


Figure 17: The laser device's Arduino and USB cable plugged into the laptop.

Go to the top-left corner of the Arduino IDE window and select the new device, “Arduino UNO”, shown in Figure 18. The serial speed in the serial monitor should change to 9600 baud. If not, change the speed manually to 9600 baud.

Note: there is no need to change the loaded software in Arduino IDE, using the software can be done entirely over serial with no need to update or re-upload code to either of the microcontrollers.

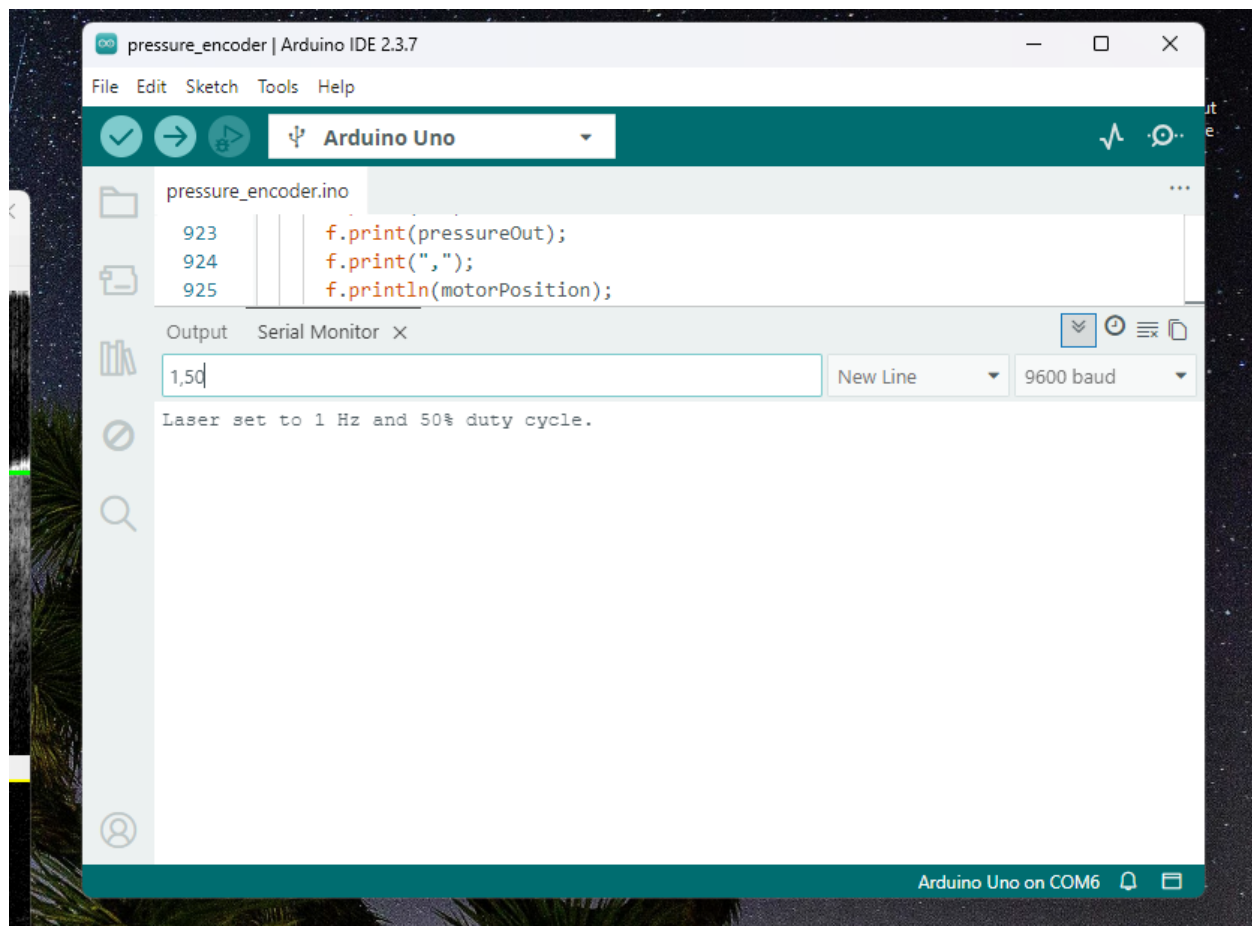


Figure 18: A connection to the laser device in the Arduino IDE window.

Enter a laser refresh rate and duty cycle (portion of a refresh that the laser is on), such as 1,50 for 1 Hz at 50% duty cycle (on and off for equal amounts of time), into the serial terminal to see a response from the laser device.

Enter any refresh rate followed by a 100% duty cycle, such as 1,100, to turn the laser on continuously. Any set of frequency and duty cycle can also be input for use in trials. Use the switch on the power box to turn the laser on and off (visible in Figure 17) and a stopwatch to track time during the trial.

After data has been successfully recorded, close both Arduino IDE and the current window of the OCT software. Open “Opxion Binary Split Program” (with a pink program icon).

Click the “Read Binary” button on the top-left of the window, and select each of the folders just generated by the OCT software (by default, the naming form begins with “raw...” and ends with

a number beginning from 0. There is one folder for every 100 frames: for example, for a 500-frame trial, select “raw...0.bin” through “raw....4.bin”, as shown in Figure 19.

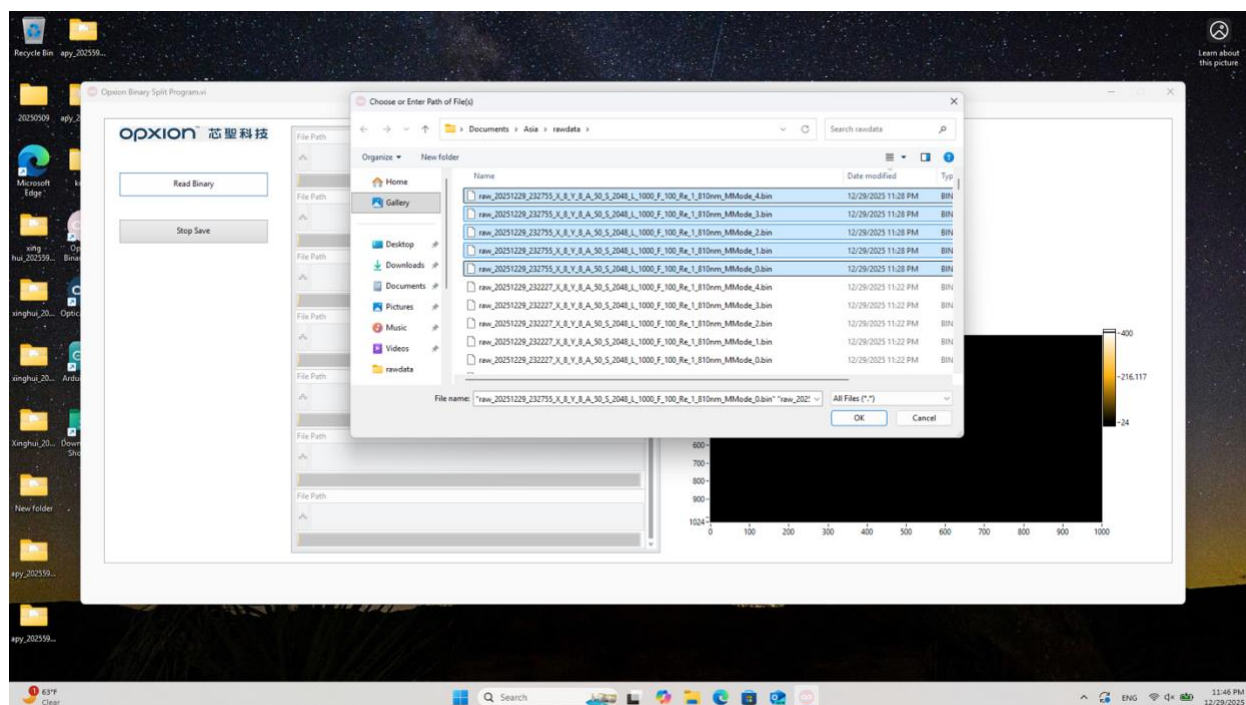


Figure 19: Loading generated OCT data into the processing software.

After your trial data is selected, press “OK” to begin processing. The software will alert you when processing is complete, and resulting image files will be placed in the “Data” folder (C:\Users\youruser\Documents\Opxion Binary Split Program\Data\) with matching names to the original raw data files (raw....bin), demonstrated in Figure 20.

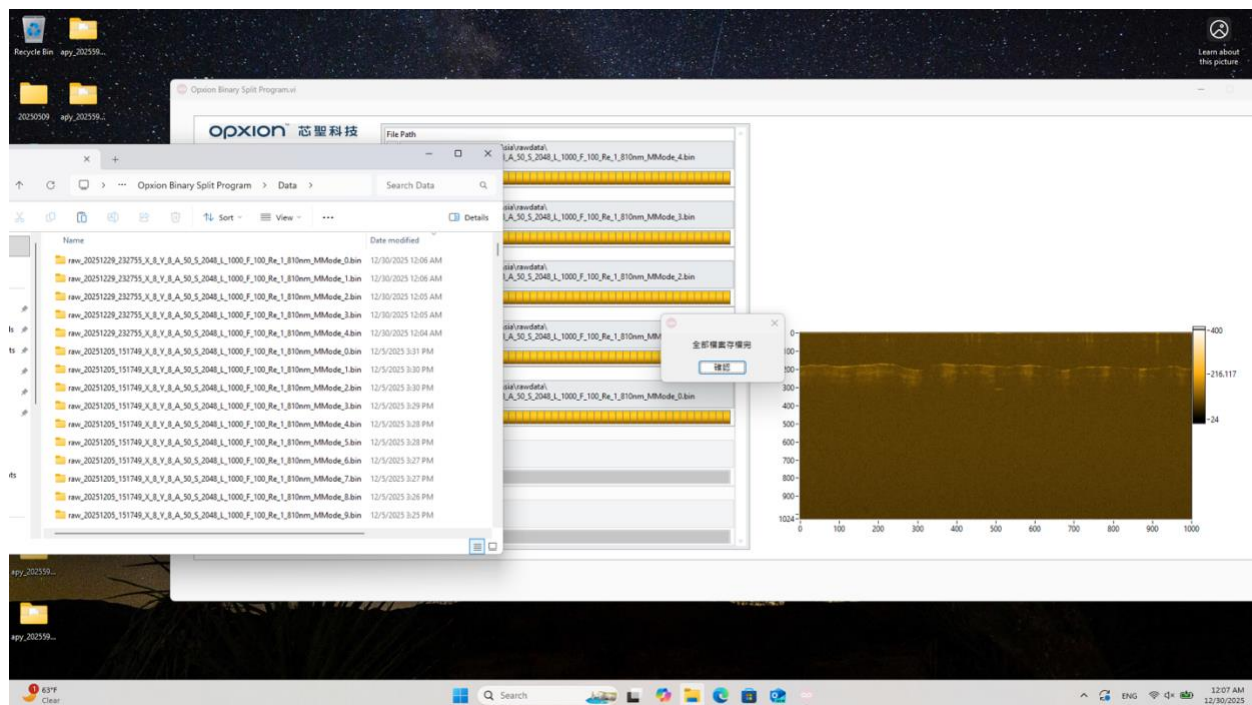


Figure 20: The OCT data processing software with image generation complete.

Once your file processing is complete, the OCT data has been generated and the trial is complete.

Note: to calibrate the device, press “cal” at the serial command prompt for the Adafruit Metro ESP32-S3. Keep the red output tube disconnected from the OCT device, with the end of your tube directly touching your finger. Follow the on-screen prompts to adjust your finger and take three high calibration points at as close to 20 kPa as possible by typing “next” into the console. Completely remove your finger afterward so that the device can take zero points with no pressure in the tubes.

Data analysis using MATLAB

MATLAB

The MATLAB testing process requires a folder structure similar to the one used with the indenter MATLAB scripts. Start by making an empty folder with each of the four MATLAB scripts inside and two folders, “data_open” and “data_save” (the same as for the indenter):

```
s1_image2csv.m  
s1_standoff_20251225.m  
s2_young_20251225.m  
s3_e_20251226_polyfit_index_k.m  
data_open/  
data_save/
```

Inside the “data_open” folder, move the output file from the /out/ folder of the SD card generated by the pressure device. Rename the CSV file to your trial name, with the letter “F” at the beginning, in the format fX_ABC: “X” is the subject number, “A” is condition 1, “B” is condition 2, and “C” is condition 3.

An example filename is f1_110.csv.

Also inside of the “data_open” folder, create another folder with the same trial name, but without a preceding letter (for example, 1_110).

Place all of the JPG files generated by the OCT device in this folder. This is easiest using a terminal program, like Windows PowerShell or Terminal on Mac. An example command for both platforms is:

```
cp folder_with_oct_data/*.bin/*.jpg folder_with_matlab_scripts/data_open/1_110/
```

This can also be done by organizing the JPG and TXT files within the individual .bin OCT output folders by type in the file window, and copying the JPG files all to the folder (/data_open/1_110 in this example). Remember to copy from each of the OCT output folders (one folder for each 100 frames).

Once completed, the folder structure with the MATLAB scripts looks like the example below.

```

s1_image2csv.m
s1_standoff_20251225.m
s2_young_20251225.m
s3_e_20251226_polyfit_index_k.m
data_open/
    1_110/
        0_1.jpg
        ...
        4_100.jpg ← Example assumes 500 total frames.
    f1_110.csv
data_save/

```

After this is complete, open the four MATLAB scripts.

First, change the directory listed at the top of the file to your working directory, and the filename listed under the “Filename” marker to your current trial, as shown in Figure 21.

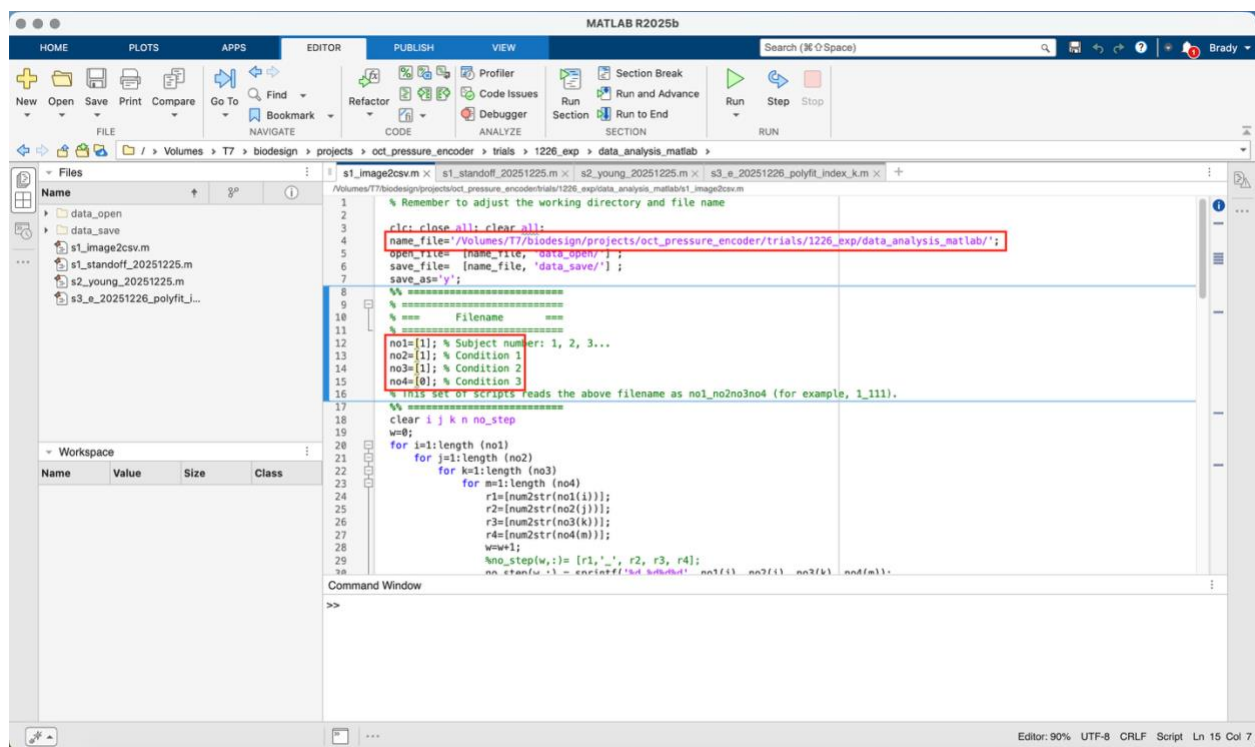


Figure 21: Change the working directory and file name as highlighted.

Once the working directory and file name are correct, press “run” to generate a sequence of images from the center columns of each individual OCT image. This will appear flipped for compatibility with following scripts, as shown in Figure 22.

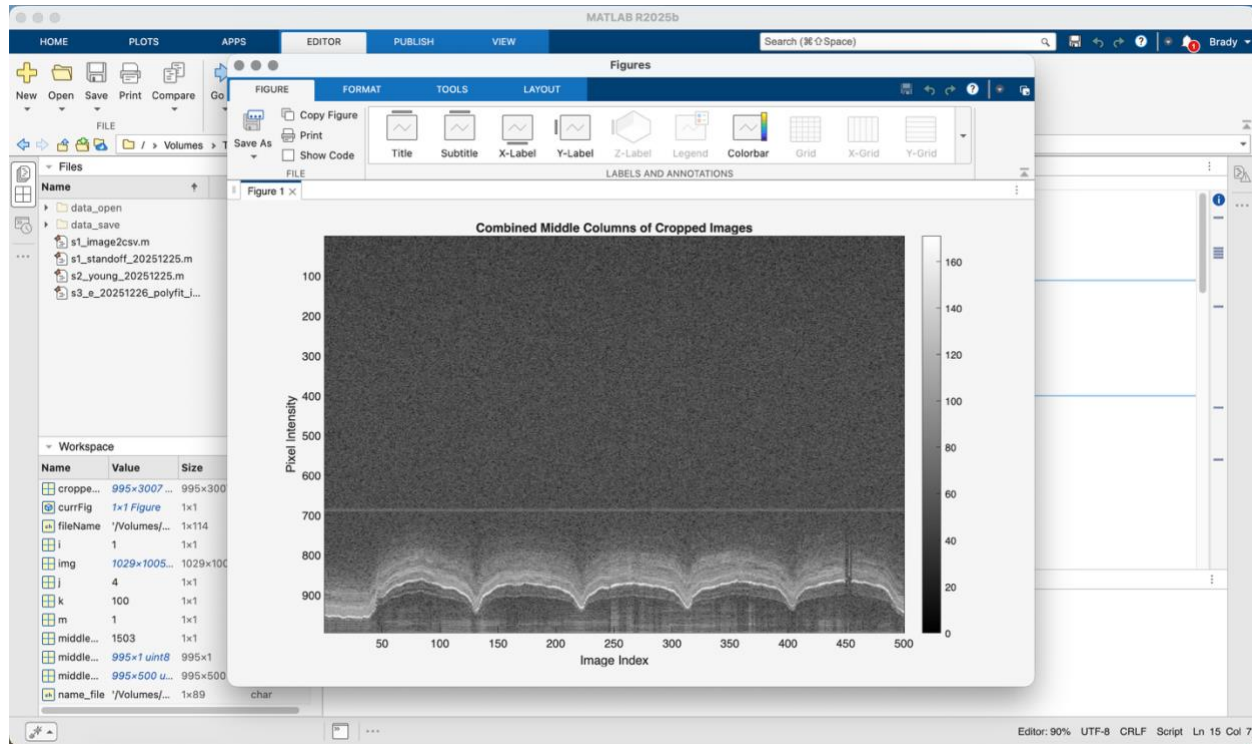


Figure 22: Output from *s1_image2csv.m*.

Once this has run, close the image window. Re-open the MATLAB working directory in your file manager and go to the “data_save” folder. The folder structure will now look like this:

```
s1_image2csv.m
s1_standoff_20251225.m
s2_young_20251225.m
data_open/
    ... Same as before
data_save/
    i1_110.csv
    i1_110.jpg
```

There are two new files in “data_save”. Move the CSV file to the “data_open” folder so that you have one folder and two files in “data_open”:

```
s1_image2csv.m
s1_standoff_20251225.m
s2_young_20251225.m
s3_e_20251226_polyfit_index_k.m
data_open/
    1_110/
        f1_110.csv
        i1_110.csv
data_save/
    i1_110.csv
    i1_110.jpg
```

Next, return to MATLAB and navigate to the tab for “s1_standoff_20251225.m”. Change the working directory and filename to the correct ones and press “run”.

A window will appear that shows a brightened version of the image from the last step. This is the tracking window: first, begin by tracking the straight line that appears above the skin deformation curve (this is used as a reference point so that the skin deformation can be calculated relative to the beginning of the skin in the visible OCT image).

Click a point on one side of the line, then another point on the other. Once done, press the “stop” button at the top-left of the window, as shown in Figure 23.

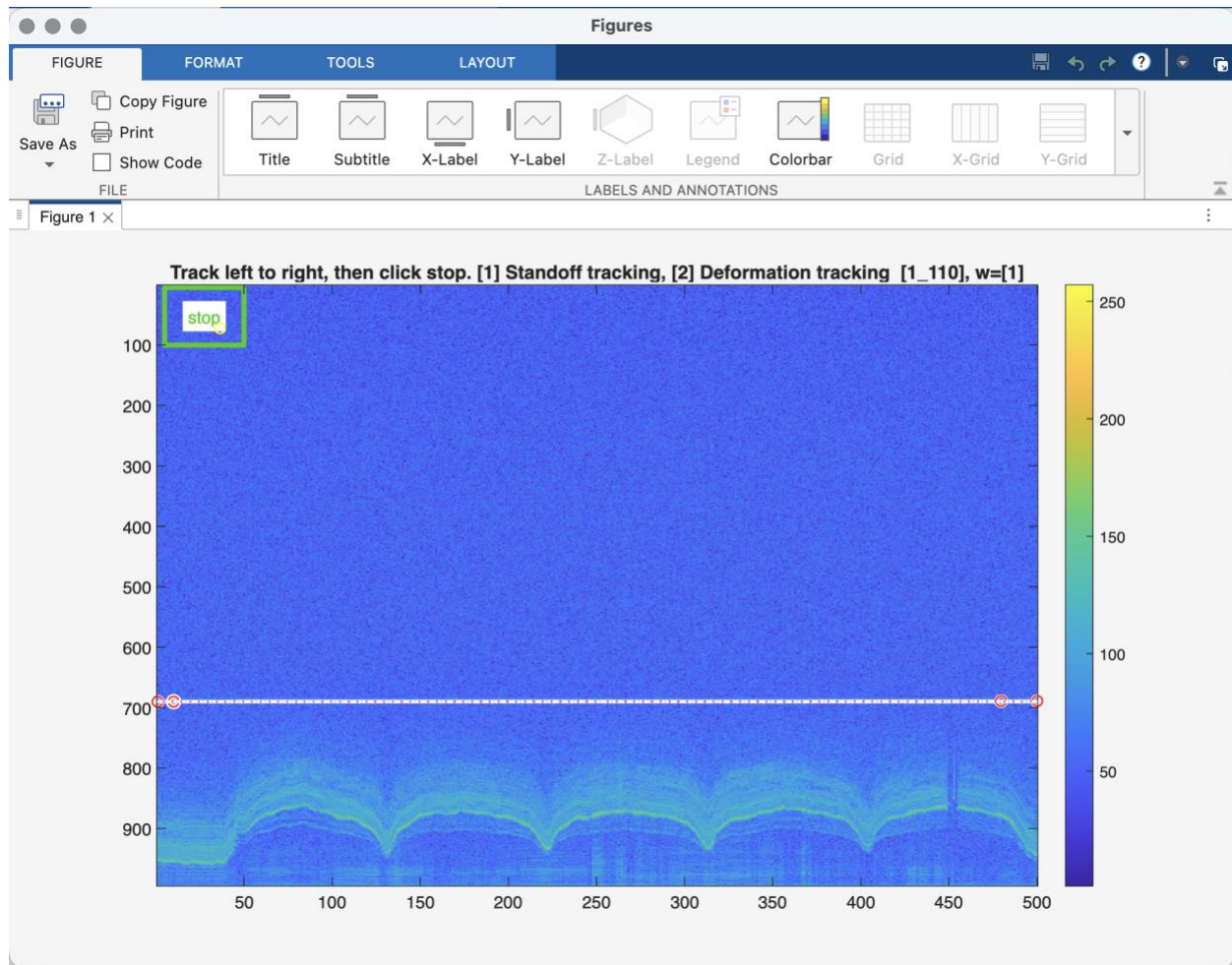


Figure 23: The tracking window from *s1_standoff*.

After this, track the skin deformation itself by beginning on the left side and clicking after each small curve in order to accurately and precisely follow the changes in height of the stratum corneum. The computer may slow down as more points are added. If a point does not appear immediately after a click, keep your cursor still until the point is generated.

When done, press the top-left “stop” button one more time, as shown in Figure 24.

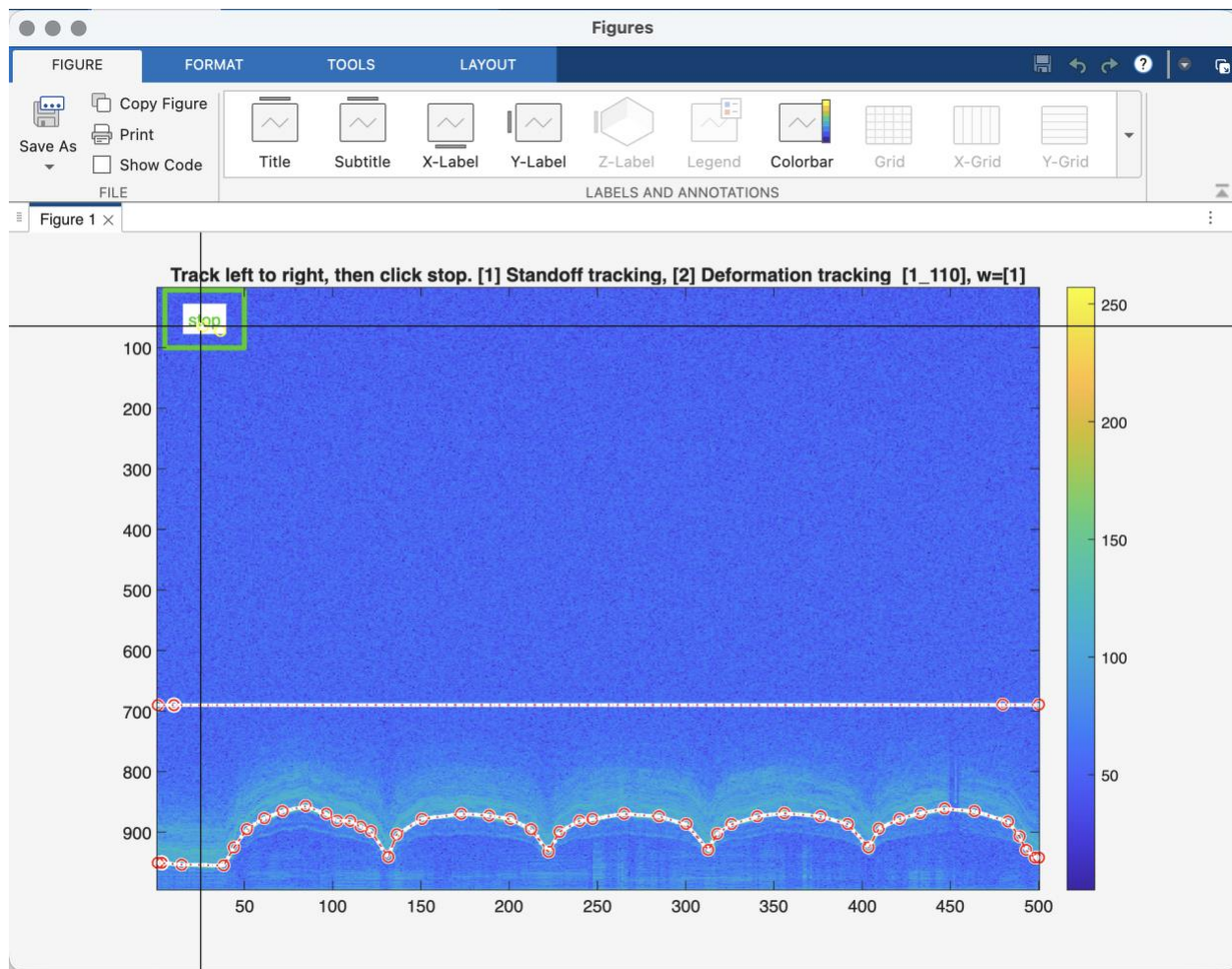


Figure 24: *s1_standoff* with tracked skin deformation.

After this, double click the “stop” button one more time. After this, the skin deformation curve with flat line reference will appear at the top of the window (flipped once more). If it does not appear immediately after the first double-click, double click again until it does.

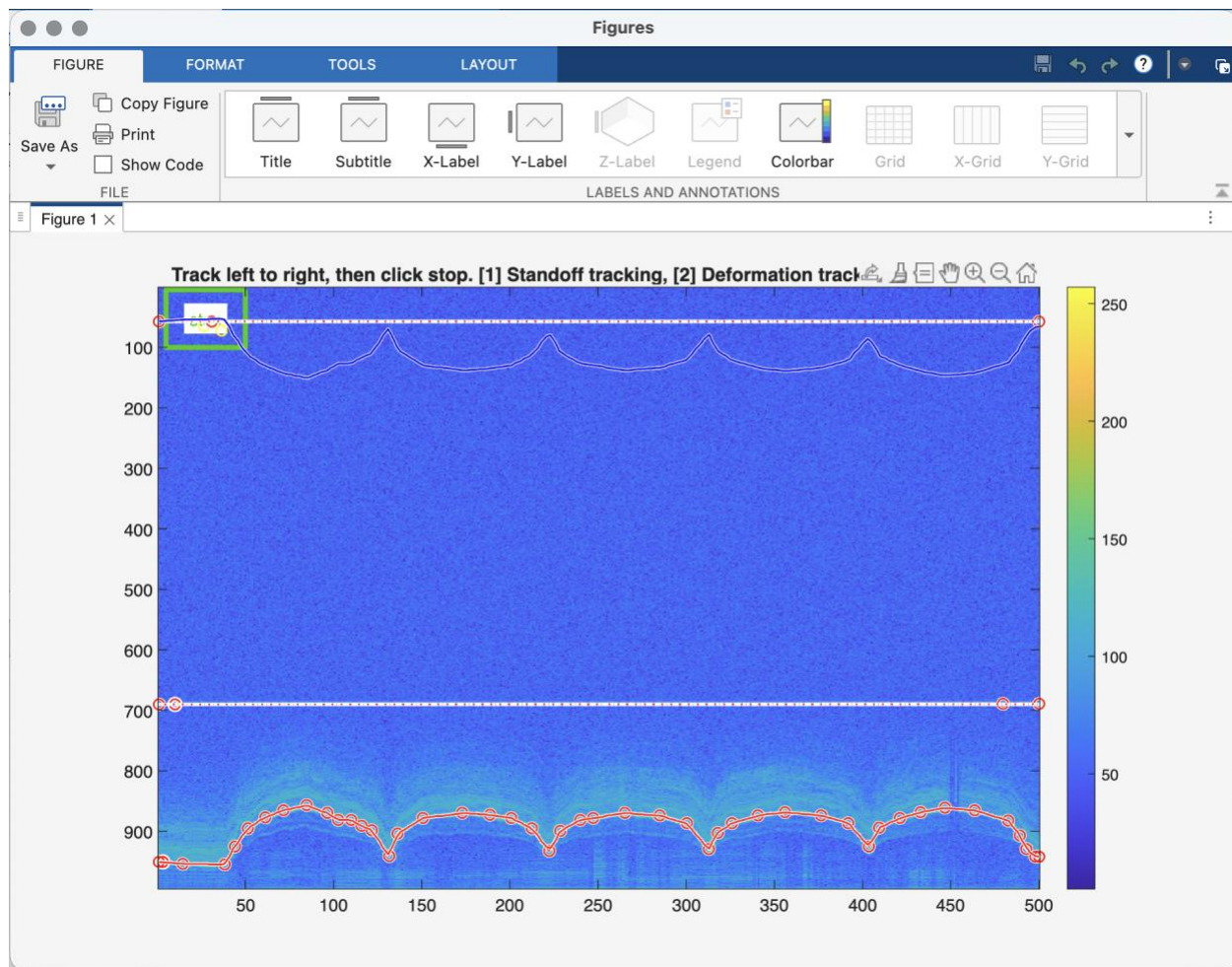


Figure 25: The finalized skin deformation tracking graph.

Once this is done, close the graph window and move to “s2_young_20251225.m”. Change the filename and working directory to the correct ones and press “run”. A window will appear as shown in Figure 26.

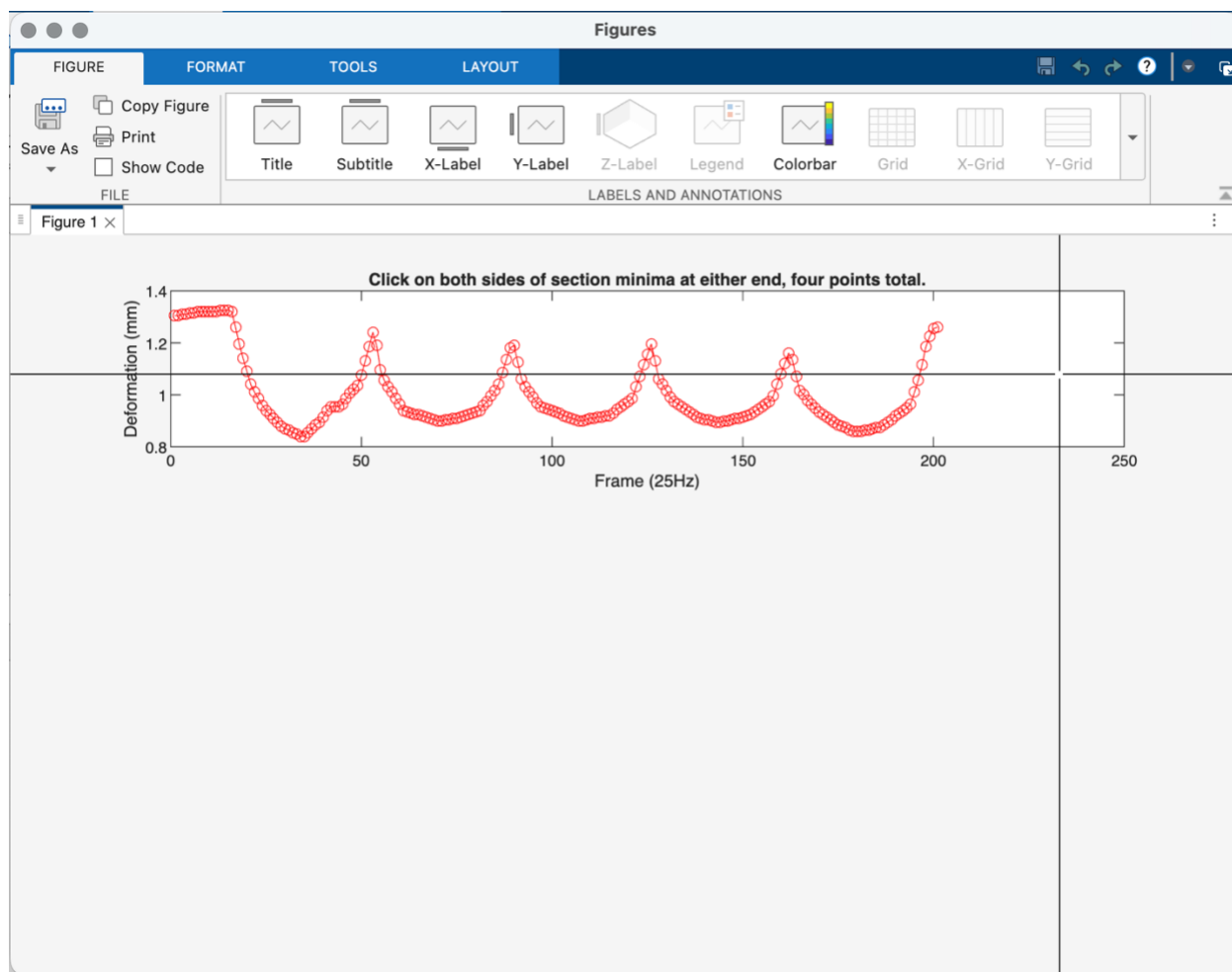


Figure 26: The empty graph from *s2_young*.

Follow the on-screen instruction to click points on either sides of the bottoms of the curves at either end of the deformation graph, for four points total. The program will automatically find the minima between both sets of points. After this, the minima will appear in green and another graph will appear, as shown in Figure 27.

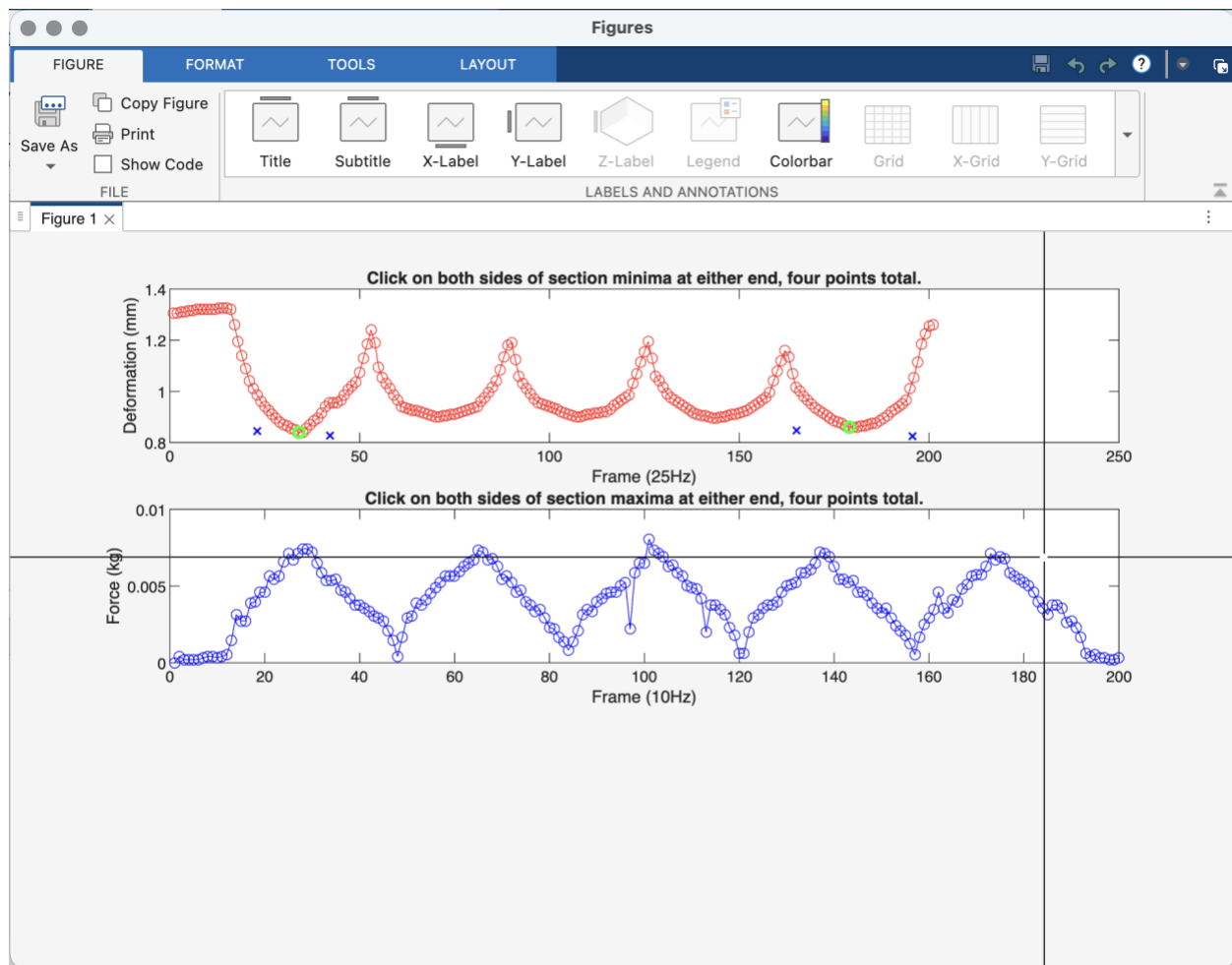


Figure 27: *s2_young* with minima selected.

Follow the instructions again for the maxima in the blue (force) graph. Select points around the same peaks as in the red graph (both of the outer peaks in most cases). After this, a force & deformation vs. time graph and hysteresis curves will appear at the bottom of the window, shown in Figure 28.

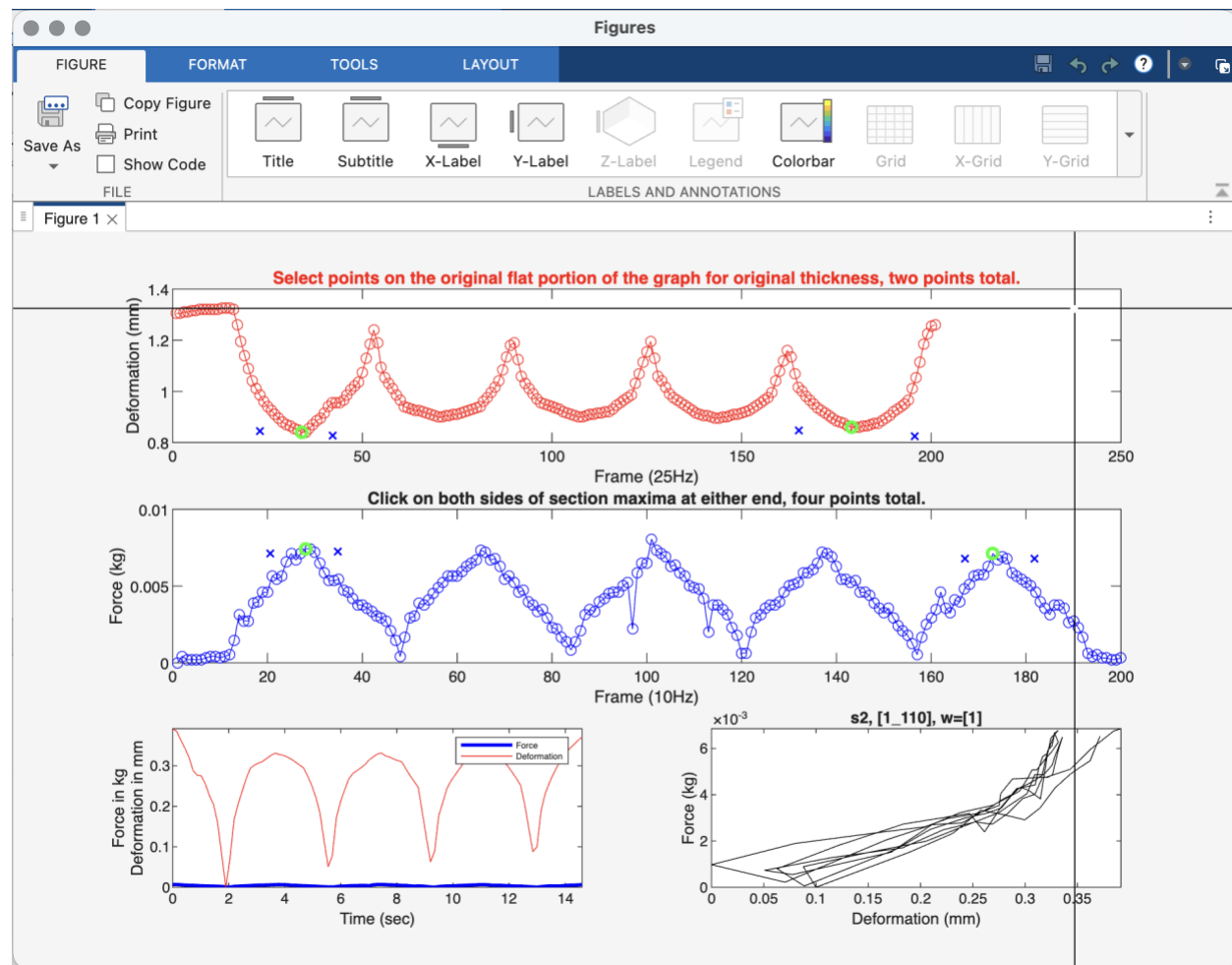


Figure 28: *s2_young* with force-deformation-time graphs and hysteresis curves.

Finally, return to the top (red) curve and select two points on the initial flat portion of the graph. The program averages the depth of these two points and uses the value as the initial thickness of the skin layer. Once this is done, close this graph window and proceed to “s3_e_20251226_polyfit_index_k.m”. Change the working directory and filename to the correct ones and press “run” to generate the final force vs. deformation graph and E1, E2, E3, and E23 Young’s Modulus values, as shown in Figure 29.

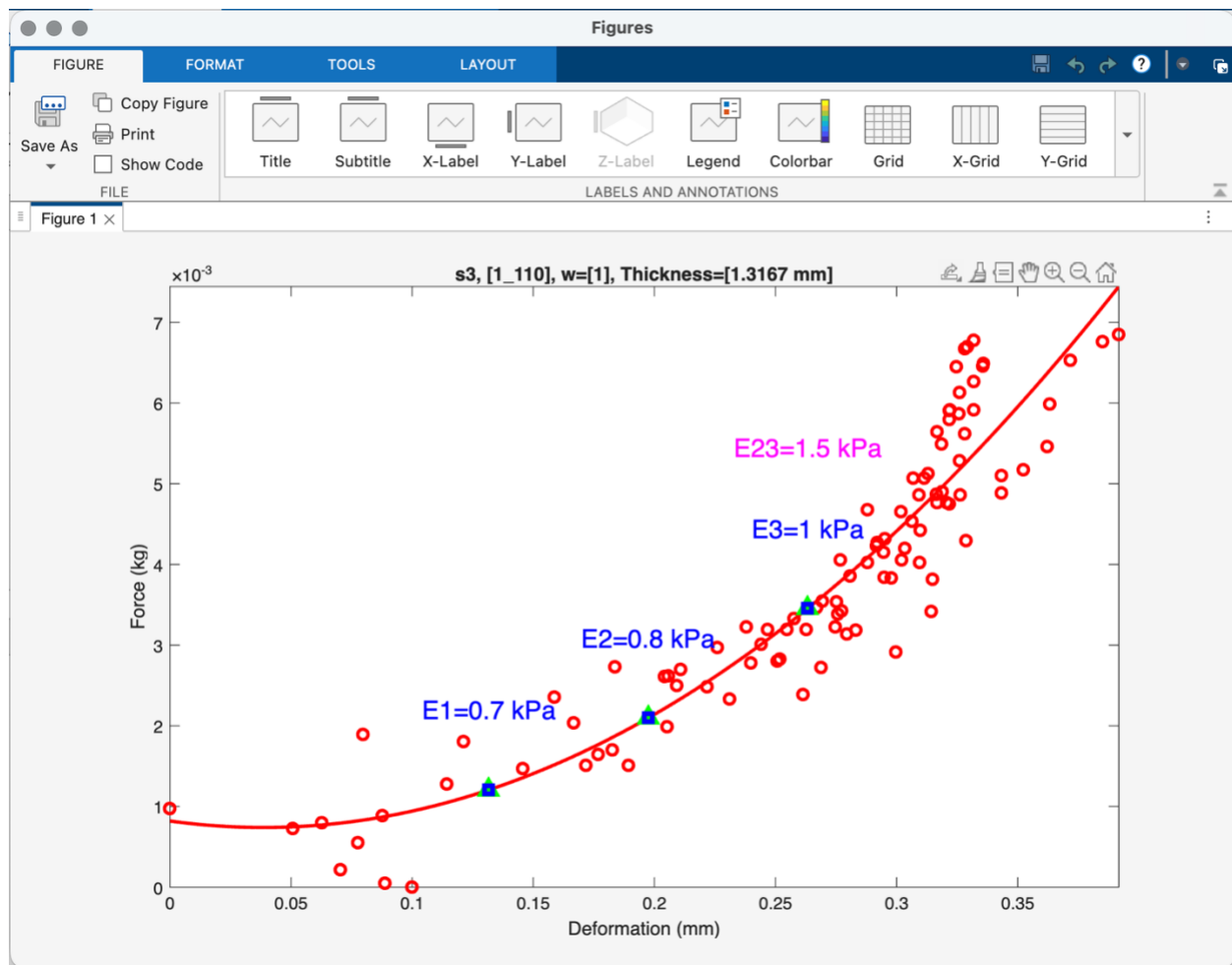


Figure 29: The final Young's Modulus graph output from *s3_e* (polynomial fit).

If necessary, the percentage deformation for calculation of E1, E2, and E3 can also be changed. Return to the MATLAB window and locate the array “ratio_h3” near the top of the file.

```
ratio_h3=[0.10, 0.15, 0.20]; % (unit: mm) % of the thickness
```

In order, the values are the percentage deformation for calculation of E1, E2, and E3 respectively. In this example, this is 10, 15, and 20 percent loading. If these values are changed to 0.05, 0.10, and 0.15, for example, the resulting graph is as shown in Figure 30:

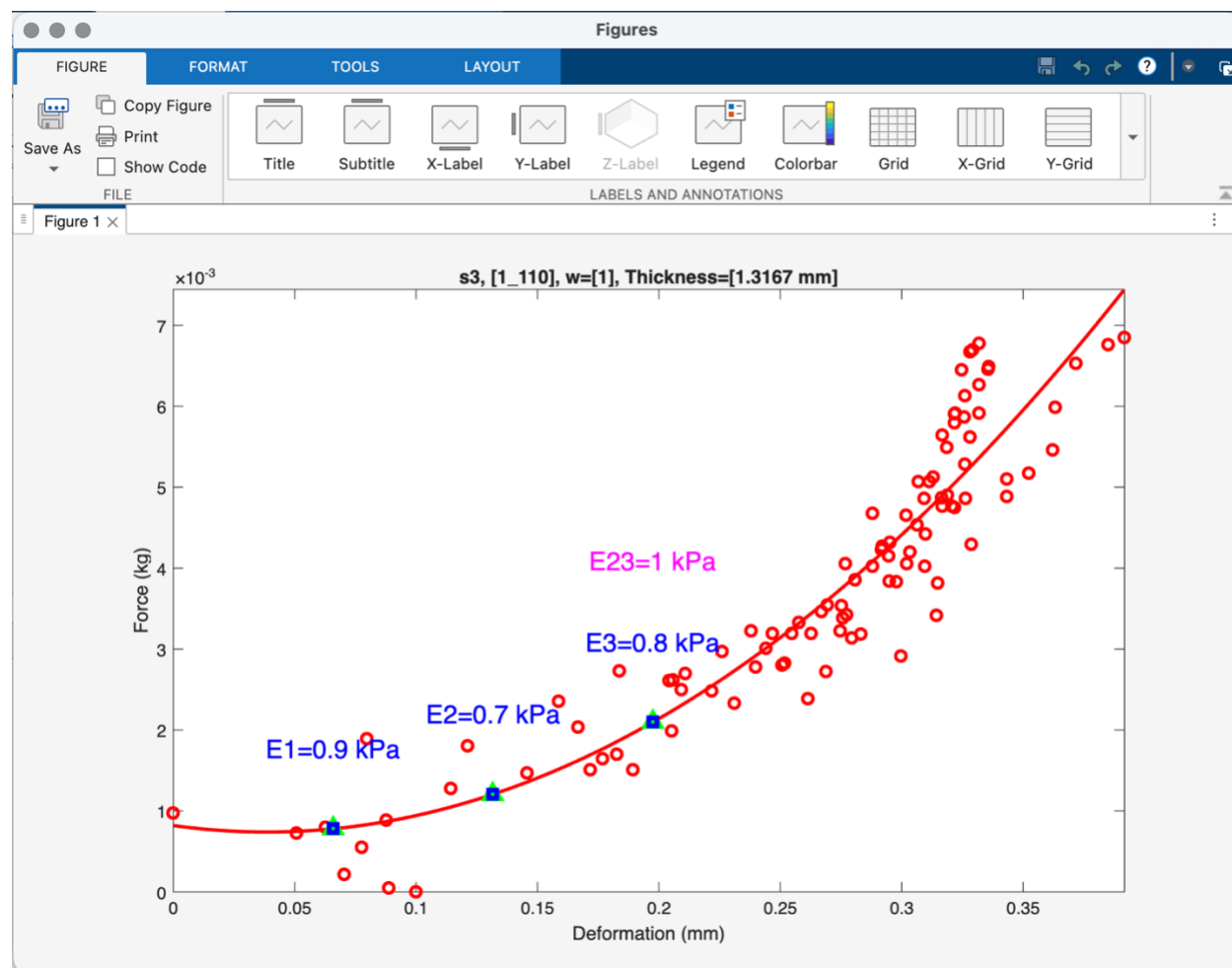


Figure 30: The $s3_e$ output graph with new $E1$, $E2$, and $E3$ loading values.

Once data processing is complete, all of these figures can also be accessed directly from the data_save folder (four total): the original image of center columns from OCT frames combined into a deformation-time graph, this graph with deformation tracking, the minima and maxima locations on the deformation-time and force-time graphs with hysteresis curves and the force-deformation-time relationship, and the final output graph with Young's Modulus values.

Pilot study

Results

For the first initial test and data collection, three data levels were taken: one control and two treatments, as shown in Table 3. The measurement location was on healthy tissue just below the left thumb (male, 20 years old), as shown in Figure 31.

Table 3: Treatment combinations.

Treatment	Laser refresh (Hz)	Laser duty cycle (%)	Duration (min)
CW0	N/A	N/A	0
PW3	1	50	3
PW6	11	50	6



Figure 31: Left hand with circled area for measurement below the thumb.

A control stiffness measurement was first taken for treatment combination CW0. After this, three minutes of a 660-nm laser operating at 1 Hz refresh and a 50% duty cycle were pointed at the measurement location for 3 minutes. After this, another stiffness measurement was taken in the same location. Following this, the same laser operating with the same parameters was pointed at the same location for another 3 minutes, and a final measurement was taken as the 6-minute measurement.

The stiffness result of the control is shown in Figure 32.

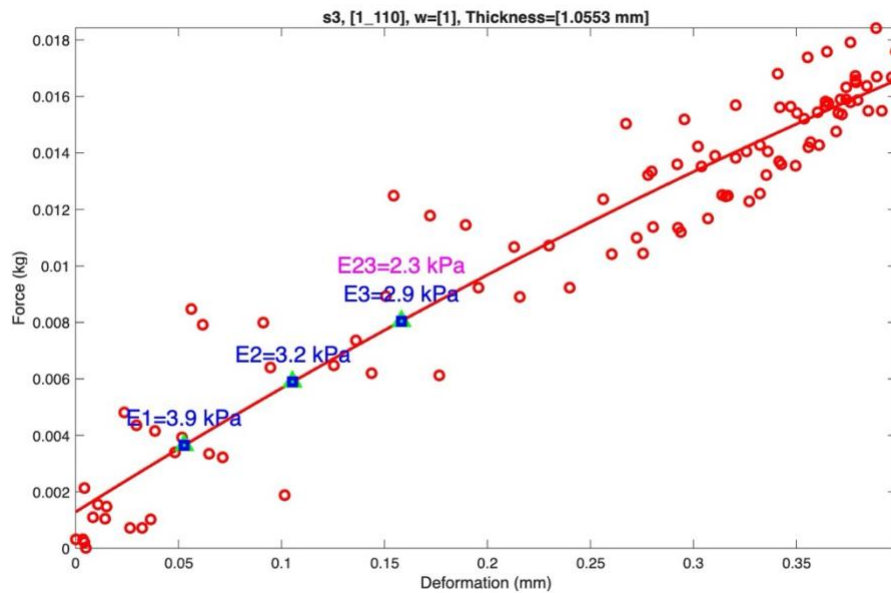


Figure 32: The stiffness result for the control measurement.

The stiffness result of the 3-minute measurement CW3 is shown in Figure 33.

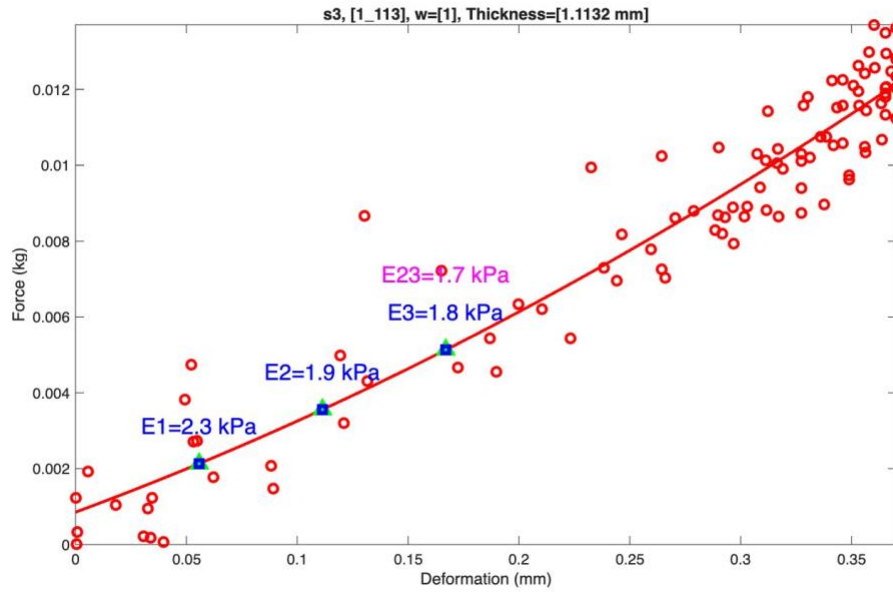


Figure 33: The stiffness result for the 3-minute laser application.

Finally, the stiffness result for the 6-minute application is given in Figure 34.

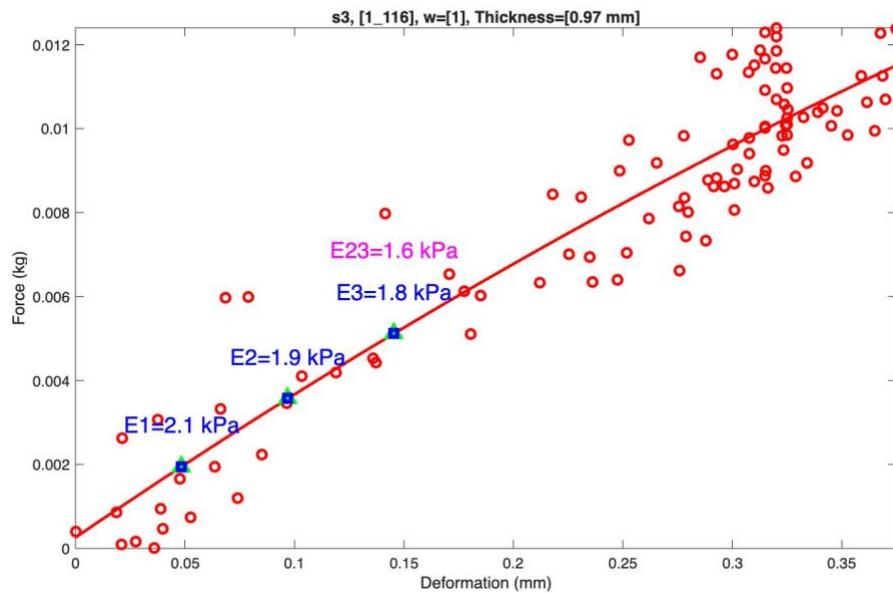


Figure 34: The stiffness result for the 6-minute laser application.

The hypothesis states that increased laser exposure decreases the stiffness of soft tissue, because the applied energy breaks surface tissue bonds. These results suggest that, while this hypothesis is supported, more stiffness decrease happens between minutes 0 – 3 than between 3 – 6.